DAVINCI: Decentralized autonomous vote integrity network with cryptographic inference

Pau Escrich[1], Marta Bellés-Muñoz[2], Jordi Piñana[1], Lucas Menéndez[1], Alex Kampa[1], Roger Baig[3], and Jose Luis Muñoz-Tapia[3]

[1] Vocdoni Association
[2] Pompeu Fabra University
[3] Polytechnic University of Catalonia

{pau,painan,lucas,alex}@vocdoni.org, marta.belles@upf.edu
{roger.baig,jose.luis.munoz}@upc.edu

Last update: July 10, 2026

Abstract. DAVINCI is the evolution of the Vocdoni voting protocol, designed to empower civil society by providing essential tools for secure, verifiable, and anonymous digital voting. Using recent advancements in zero-knowledge (ZK) proof systems and blockchain technology, DAVINCI transitions to a specialized ZK rollup system that inherits network security from settlement layers such as the Ethereum mainnet. The system relies on cryptographic proofs to ensure integrity and security, eliminating the need for centralized authorities. By integrating ZK-SNARKs and threshold homomorphic encryption, DAVINCI enables end-to-end verifiability, privacy, and trustlessness in election processes. The protocol distributes the election key among a set of key wardens through distributed key generation, coordinates the process through Ethereum smart contracts, and uses Ethereum data blobs for data availability. With a focus on accessibility, scalability, receipt-freeness, and automation, DAVINCI aims to facilitate high-frequency and low-cost voting, fostering mass adoption of voting and simplifying civil participation. Finally, the design of DAVINCI is grounded in practicality: all components are implemented using current technologies and evaluated on production-grade hardware. In sum, the proposed architecture is not merely theoretical, but a viable solution ready for short-term deployment. The source code has been made open source, allowing practitioners and researchers to further investigate its details and potential for future uses.

Table of Contents

1 Introduction

Online voting remains one of the most studied yet elusive applications in applied cryptography. As digital services expand across both public and private sectors, secure and universally verifiable online voting systems have become an increasingly desirable goal. Remote electronic voting promises scalability, accessibility, and administrative efficiency; yet the design of systems that simultaneously ensure privacy, integrity, coercion resistance, and transparency under realistic adversarial models remains a formidable challenge. Numerous deployments have revealed deep-rooted usability flaws and security gaps, particularly when verifiability mechanisms are either poorly understood by users or reliant on centralized infrastructure.

Against this backdrop, the Vocdoni project was initiated in 2018 with the aim of rethinking online voting from first principles. The name Voĉdoni, meaning *to give voice* in Esperanto, reflects the project's foundational goal: to empower collectives, from small associations to millions of citizens, to engage in secure and verifiable decision-making, regardless of technological or institutional barriers. Central to this vision was the idea that voting is not limited to formal governmental elections but serves as a more general-purpose mechanism for collective signaling. Vocdoni introduced a fully anonymous end-to-end verifiable voting system designed to operate efficiently on a range of devices, including smartphones. To support these goals, the team deployed a custom infrastructure emphasizing resilience, neutrality, and transparency.

Technically, the architecture of Vocdoni was based on a bespoke byzantine fault tolerant (BFT) layer-1 blockchain, named Vochain. At the time, efficient zero-knowledge (ZK) proof systems were emerging, but not yet practical for most deployments. Vochain provided a performant and low-cost environment (achieving approximately 700 transactions per second) in which advanced cryptographic tools could be used without the constraints imposed by general-purpose blockchains based on the Ethereum virtual machine (EVM). The ability to issue voting transactions without requiring user fees enabled broader accessibility. Over several years of development and deployment, this architecture proved both viable and valuable in practice. However, broader adoption as a universal voting protocol highlighted the need for further architectural refinements and stronger formal guarantees.

In this work, we introduce DAVINCI, a new protocol that builds upon the lessons and conceptual groundwork laid by Vocdoni. DAVINCI adopts a modular design and integrates state-of-the-art cryptographic tools, including succinct ZK proofs, improved bulletin board constructions, and robust coercion resistance mechanisms, reflecting the most recent advances in academic research. Unlike monolithic designs, DAVINCI is conceived as a composable primitive: a foundational layer intended to support secure and verifiable voting in diverse contexts, from blockchain governance to institutional elections. This shift in design philosophy aims to address long-standing challenges in the field, offering a cleaner abstraction with clearer security boundaries and formal underpinnings.

1.1 Contributions

This work introduces DAVINCI, a modular and verifiable protocol for digital voting, designed to support privacy-preserving, auditable, and adaptable election processes across diverse governance contexts. The protocol *models voting as a constrained state machine*, in which each valid operation is expressed as a formally defined state transition, enforced through succinct ZK proofs. That is, at the core of the system lies a set of reusable arithmetic circuits that implement voter authentication, encrypted ballot validation, state transitions, and tally finalization. These circuits are optimized for off-chain proving and are compatible with modern zero-knowledge succinct non-interactive argument of knowledge (ZK-SNARK) protocols. Application-layer state is maintained off-chain and committed on-chain using Merkle trees as cryptographic accumulators.

A key contribution of this work is the introduction of a generic ballot protocol that abstracts a wide range of voting systems into a unified, parameterized representation. Instead of designing specialized circuits for each voting rule, ballots are expressed as fixed-length vectors subject to configurable constraints, enabling support for approval, ranking, quadratic, single-choice, and multiple-choice elections within the same circuit framework. This design significantly reduces circuit complexity while allowing different tallying and counting rules to be enforced efficiently inside ZK proofs.

To coordinate protocol execution over a decentralized network of nodes, DAVINCI includes a set of Ethereum smart contracts that mediate the setup of elections, manage finalization procedures, and verify

submitted proofs of correctness. A standardized transaction and data encoding format supports all stages of the process, including vote casting, ballots aggregation, and result publication. This approach ensures that protocol interactions remain deterministic and interoperable across clients and backends. The system also introduces the Vocdoni token, which is used to align incentives between participants. Altogether, the design aims to balance auditability and privacy while reducing trust assumptions on infrastructure providers. By cleanly separating functionalities and providing modular interfaces, the protocol is intended to serve as a foundation for both practical deployments and future extensions, such as integration with alternative identity systems, off-chain data availability layers, or other consensus backends.

1.2 Related work

Foundations of cryptographic voting. Cryptographic e-voting originated with Chaum's mix-net construction [15], which achieves anonymity through verifiable shuffling of ciphertexts. Helios [1] was the first deployed web-based system to combine threshold ElGamal encryption with a public bulletin board and homomorphic tallying, and it remains a standard reference point for end-to-end (E2E) verifiability in practice. Belenios [16] refines this design with formal verifiability proofs and a clean separation between the registration authority and the voting server; the BeleniosRF variant additionally achieves receipt-freeness, but only under the assumption that the voting server is honest regarding that property. ElectionGuard [5] belongs to the same family, combining homomorphic tallying with threshold guardians, publicly verifiable proofs, and a centralized election administrator that drives the tally. A common feature of all of these systems is that, beyond their cryptographic core, they require a non-trivial setup of either the bulletin board, the tally authority, or both, and that the operator set is fixed for each election rather than open to participation.

Receipt-freeness and revoting. A receipt-free voting system prevents a voter from producing convincing evidence of their vote, mitigating bribery and simple coercion. Benaloh and Tuinstra [6] gave the first formal notion, and Hirt [24] showed how to achieve it efficiently by combining homomorphic encryption with re-encryption performed by a designated helper. Doan et al. [17] subsequently generalize Hirt-style receipt-freeness to a setting with multiple ballot-processing servers, removing the single-point-of-failure assumption common to earlier non-interactive constructions. These constructions remain server-based: none of them places the protocol on a public bulletin board with cryptographically enforced data availability, and the helpers responsible for re-encryption are selected in advance rather than drawn from an open set.

Blockchain-based and ZK-based voting. McCorry et al. [30] proposed the first self-tallying voting contract on Ethereum, but placing every vote directly on-chain is expensive and creates a linkable trail between voters and their choices. Permissioned-ledger proposals [29] improve operational properties but still rest on honest-majority assumptions over a closed set of validators. Freedom Tool [33] uses ZK proofs over biometric passports to enforce eligibility, but votes appear on-chain in cleartext, trading ballot secrecy for transparency. MACI [12] uses ZK proofs to discourage bribery, but its coordinator is a single trusted party that can read individual votes; although later versions improve usability, this assumption remains. Semaphore [23] provides anonymous group-membership proofs and is reused as a building block in several voting prototypes, although it is not itself a complete voting protocol. Cicada [20] and Shutter Network [32] use timelock and threshold encryption to keep ballots hidden until the tally, but neither re-randomizes stored ciphertexts after submission. As a result, voters can still prove how they voted, weakening protection against coercion and vote-selling.

1.3 Paper organization

The rest of the paper is organized as follows. In Section 2 we introduce the necessary background. In Section 3 we provide a high-level overview of a voting process, omitting technical details for clarity. In Section 4 we describe the full voting process in detail and in Section 5 we focus on the ballot protocol. In Section 6 we analyze the protocol, covering its security properties, implementation details, and performance evaluation. In Section 7 we conclude the paper with final remarks and future work. Finally, we include Appendix A with the concrete instantiations of all the cryptographic primitives used in the protocol.

2 Background

The DAVINCI protocol builds on several decentralized technologies and advanced cryptographic tools. In this section, we give a high-level overview of these components and their role in the system. We first introduce the interplanetary file system (IPFS), which serves as a decentralized storage layer for distributing large datasets off-chain. We then describe Ethereum, the settlement layer where commitments are published, rules are enforced through smart contracts, and state data is managed using recent mechanisms such as data blobs. Finally, we present the ZK proof systems that underpin the protocol's integrity and security, focusing on ZK-SNARKs and arithmetic circuits, which allow participants to prove compliance with protocol rules without revealing sensitive information.

2.1 Interplanetary file system

The interplanetary file system (IPFS) is a peer-to-peer distributed storage network designed to make the web more resilient, permanent, and decentralized. Unlike traditional client-server architectures that retrieve data from a specific location (e.g., a URL pointing to a server), IPFS retrieves data based on its content. Every file stored in IPFS is split into blocks, each block is hashed, and the resulting cryptographic hash is used as its unique identifier. This property, known as content addressing, ensures that data is tamper-evident: if the file changes, so does its hash. IPFS also provides deduplication (the same content is only stored once across the network), versioning (content identifiers can point to immutable snapshots of data), and distributed availability (data can be fetched from any node that stores it). Together, these features allow IPFS to function as a decentralized content delivery network.

In the context of the DAVINCI voting protocol, IPFS is particularly useful for off-chain data distribution. Large datasets such as the full census (eligible voters and their weights) and election metadata (which may include questions, options, images, etc.) do not need to be stored directly on Ethereum. Instead, they are stored in IPFS, and only their content hashes are published on-chain. This approach ensures transparency and verifiability, as anyone can fetch the dataset from IPFS and recompute the hash to confirm its integrity, while keeping on-chain storage costs low. That is, election participants can therefore rely on IPFS to access the data without burdening the blockchain with large amounts of data.

2.2 Ethereum

Ethereum is a decentralized blockchain platform that supports programmable transactions through its built-in execution environment, the Ethereum virtual machine (EVM). All interactions with the Ethereum network take the form of transactions, which must be broadcast to the network, validated by consensus, and permanently recorded on-chain. Every transaction requires the payment of a fee (measured in units of gas), denominated in ETH, to compensate validators for computation and storage. This fee model ensures that resources are used efficiently and prevents denial-of-service attacks by making large or complex operations costly. In the context of DAVINCI, Ethereum serves as the settlement layer where critical commitments are published, ensuring transparency and immutability.

Smart contracts. Smart contracts are programs deployed on the Ethereum blockchain that execute deterministically in response to transactions. Once deployed, they cannot be altered, and their execution is guaranteed by the consensus of the network. In DAVINCI, smart contracts orchestrate the election by managing state transitions and storing critical data such as the current state root and the encryption public key. They enforce the protocol rules in a trustless environment, ensuring that no single participant can manipulate the election. By anchoring these rules in Ethereum smart contracts, DAVINCI guarantees that the election logic is applied consistently and verifiably across all participants.

Data blobs. Data availability is a key requirement for decentralized protocols. Ethereum's recent EIP-4844 [13] introduces data blobs as a mechanism for publishing large volumes of off-chain data alongside transactions at a lower cost than traditional storage. These blobs are kept on-chain for 4096 epochs, approximately 18 days, after which they are pruned. For longer election periods, third-party solutions based on EIP-4844 can be used to extend data availability. In DAVINCI, sequencers use data blobs to share state

transition data efficiently. By publishing state updates in blobs, sequencers allow other participants to reconstruct and verify the evolution of the election without overloading Ethereum's permanent storage. This ensures scalable, decentralized data availability while retaining verifiability.

2.3 Merkle trees

Merkle trees [31] are cryptographic hash trees that enable compact proofs of data inclusion. Formally, a Merkle tree is a binary tree structure where each leaf node represents the cryptographic hash of a set of data and every non-leaf node is derived from the hash of its children. The apex of the tree, the *Merkle root*, acts as a succinct cryptographic commitment to the entire dataset. To prove that a specific element x exists within the set, a prover provides a Merkle path consisting of the sibling nodes along the path from the leaf to the root. A verifier can reconstruct the root in logarithmic time $O(\log n)$ and check it against the committed state, without requiring access to the full dataset. In DAVINCI, we use two specialized variants of this structure to commit critical data: a sparse Merkle tree for the voting state and an incremental Merkle tree for the voter registry (census).

Sparse Merkle trees. A sparse Merkle tree (SMT) represents a key-value map where each leaf is the hash of a (key, value) pair and each parent node is the hash of its two children. In an SMT, every possible key maps to a unique leaf position (with empty slots defaulting to a zero value), yielding a fixed-height binary tree. This property is circuit-friendly, since the proof of a leaf's membership or non-membership has a consistent length and can be efficiently verified inside a ZK-SNARK circuit using a SNARK-optimized hash. In DAVINCI, the state tree is implemented as an SMT of fixed depth. In particular, configuration parameters occupy reserved leaf addresses, each voter's encrypted ballot is stored at a leaf indexed by their unique identifier, and each one-time vote identifier is recorded at a dedicated leaf path. Because the SMT covers a vast key space, a user or sequencer can prove that a given key is present in the state (or conversely, that it remains empty) by providing a short Merkle proof against the tree's root, rather than revealing the entire state. This enables compact proofs of state correctness. For instance, showing in ZK that a vote identifier has not appeared before or that a new ballot's ciphertext is correctly inserted into the state. The SMT used in DAVINCI follows the circomlib Merkle tree specification (Poseidon-based), ensuring that state updates can be verified succinctly on-chain via the Merkle root and proof.

Incremental Merkle trees. The census (voter registry) is maintained with an incremental Merkle tree (IMT), a binary Merkle tree designed for efficient sequential updates. Unlike a sparse tree, the IMT grows in height only as needed with the number of leaves and eliminates the overhead of hashing dummy "zero" siblings. In this scheme, voters are assigned sequential leaf indices in a continuously evolving tree, rather than being placed at cryptographic key-derived positions. In DAVINCI, the census Merkle tree is built as an IMT: each eligible voter's identifier (or a commitment to it) and voting weight are stored in the next available leaf, and only the Merkle root of this tree (the `censusRoot`) is published on-chain. This design makes membership updates and proof generation extremely low-cost for the census. Appending a new voter requires recomputing hashes only along the single path from the new leaf to the root, and the tree's depth adjusts optimally (e.g. a tree of N voters has height $\approx \log_2 N$, instead of a fixed large height). Consequently, proving one's inclusion in the voter list is more efficient than it would be with an SMT: the Merkle proof is shorter and involves no unnecessary default nodes. A voter can thus obtain a small proof of their membership in the census (their leaf's existence under the known root) without revealing their index or identity, and include this in a ZK-SNARK to demonstrate eligibility. In summary, the IMT's focus on sequential insertion and minimal hashing overhead makes it well-suited for the frequently-updated census tree, offering better update performance and smaller circuits for verification than a traditional SMT structure in this context.

2.4 Zero-knowledge proofs

Zero-knowledge (ZK) proofs are cryptographic protocols that allow one party (the prover) to convince another (the verifier) that a certain statement is true, without revealing any information beyond the validity of the statement itself. In the context of voting, this enables participants to prove that their encrypted ballot is well-formed and complies with the election rules, without disclosing their actual choice.

ZK-SNARKs. We focus on a specific type of proof system called zero-knowledge succinct non-interactive arguments of knowledge (ZK-SNARKs). Informally, while a ZK proof convinces the verifier that a valid witness exists, an argument of knowledge additionally ensures that the prover actually knows such a witness, except with negligible probability. ZK-SNARKs extend this notion with two crucial properties: they are non-interactive, requiring only a single message from prover to verifier, and succinct, meaning that proof size and verification cost remain small regardless of the size of the underlying statement. For instance, Groth16 proofs [22], which rely on pairing-based cryptography over elliptic curves, are only about 200 bytes long and can be verified in a few milliseconds. These properties make ZK-SNARKs particularly well-suited for blockchain applications, where verification cost and data size are critical. In DAVINCI, ZK-SNARKs are used at multiple stages of the protocol: voters generate proofs to show that their encrypted ballots are correct and sequencers use recursive ZK-SNARKs to allow multiple proofs to be aggregated efficiently and generate proofs to attest to valid state transitions. In essence, ZK-SNARKs enable both voters and sequencers to produce compact proofs that can be checked on-chain, ensuring that all protocol rules are followed without requiring trust in any party.

Trusted setup. A limitation of some ZK-SNARK protocols is their reliance on a common reference string (CRS) generated during a so-called trusted setup. The CRS is built from random values that must not be known to either the prover or the verifier. To achieve this, the CRS can be created by a trusted third party or, more robustly, via a secure multi-party computation (MPC) protocol [14]. In an MPC setup, it suffices that at least one participant discards their secret contribution to ensure the integrity of the whole ceremony.

Arithmetic circuits. In general, ZK-SNARKs operate by proving the satisfiability of an arithmetic circuit. An arithmetic circuit (from now on also called circuit or ZK circuit) is a directed acyclic graph where nodes represent addition or multiplication gates over a finite field. The inputs and outputs of the circuit correspond to the public inputs and private witnesses of the computation, while the intermediate wires carry intermediate values. Any deterministic computation, from verifying an encryption to checking Merkle proofs, can be compiled into an arithmetic circuit [19]. In practice, proving with ZK-SNARKs means showing knowledge of a secret witness (e.g., a plaintext ballot or encryption randomness) that satisfies all the equations encoded in the circuit. For example, the ballot circuit in DAVINCI encodes the constraints that a ciphertext is well-formed, that the vote identifier is correctly derived, and that the ballot complies with the rules of the voting mode. The prover generates a proof of satisfiability, while the verifier only checks the proof against the public inputs, without learning the secret witness. By building complex protocol logic from arithmetic circuits and using ZK-SNARKs to prove their satisfiability, DAVINCI ensures that every step of the election process, from ballot submission to state transitions, is enforced cryptographically and publicly verifiable.

3 Protocol intuition

The following section provides an overview of the DAVINCI protocol, describing its main phases and actors without delving into technical details. An election is governed by smart contracts deployed on Ethereum, which ensure transparency, enforce rules, and verify proofs. As shown in Fig. 1, the protocol consists of five main phases, from the setup of the election to the publication and verification of the final results.

1. *Election setup.* In the first phase, the *organizer* gathers all census data and generates a cryptographic commitment to this data. Additionally, the organizer defines the voting parameters, including the number of voting options and the vote-counting mechanism (e.g. weighted voting, quadratic voting, etc.). Once these details are set, the organizer submits a transaction to the process management smart contract on the Ethereum blockchain. This transaction publicly records the voting parameters and census commitment, ensuring transparency and preventing any subsequent alterations to the voting setup.

2. *Encryption key generation.* A designated group of participants, referred to as *key wardens*, collaboratively generate a shared encryption public key, which voters will use to encrypt their votes. This key is established through a distributed key generation (DKG) protocol, ensuring that no single party can control or reconstruct the corresponding private key independently. Once the encryption public key is securely computed and published in the smart contract, the voting period can start.

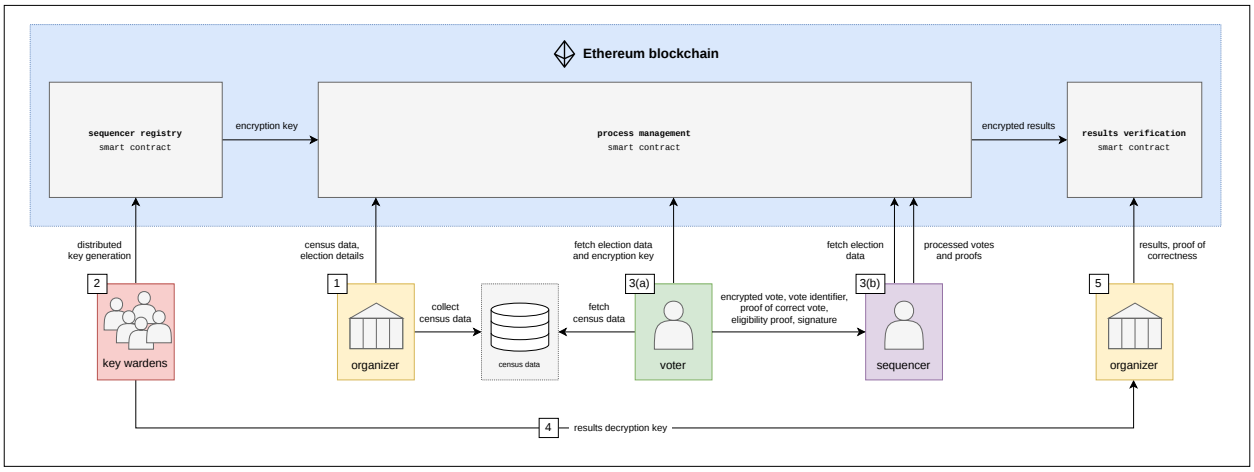

Fig. 1: Overview of the protocol flow.

3. *Voting period.* During the voting period, two actors interact continuously: *voters*, who cast their encrypted ballots, and *sequencers*, independent entities responsible for collecting, verifying, and processing these ballots before committing them to the shared state. Both processes, (a) ballot submission by voters and (b) vote batching by sequencers, occur continuously and in parallel throughout this phase. The voting period remains open until the deadline defined by the organizer, allowing voters to submit or overwrite ballots at any time, while sequencers periodically process new submissions and update the election state accordingly.

   (a) *Vote casting.* Voters select their preferred choices according to the voting rules established by the organizer and captured in the process management smart contract. Instead of submitting their votes directly on-chain, they send them off-chain to a sequencer of their choice for processing. To ensure privacy, votes are encrypted using the available encryption public key. Additionally, voters compute a unique vote identifier that will allow them to verify that their vote has been included in the final result. Alongside the encrypted vote and the vote identifier, each voter must also provide the following data: a proof of valid voting, demonstrating that the vote complies with the election rules; a proof of eligibility, verifying that they are registered in the census; and a proof of identity ownership, in the form of a digital signature, confirming that they are the legitimate voter and are not impersonating someone else. To mitigate coercion and vote-buying, the protocol allows voters to overwrite their vote any number of times during the voting phase.

   (b) *Vote batching.* During this phase, sequencers collect encrypted votes from multiple voters along with their corresponding proofs, and verify the validity of these submissions. That is, sequencers verify the signatures, to ensure the votes were cast by legitimate voters; they verify the proof of compliance, confirming that each vote adheres to the election voting rules, and the census membership proofs against the commitment to the census data that was originally registered in the process management smart contract. Once the sequencers have processed and verified all votes, they must prove that these verifications were performed correctly. Instead of submitting individual verifications for each vote, they generate a single ZK proof that attests to the correctness of all verifications. Additionally, the sequencers reencrypt the votes and generate a proof of the correct reencryption computation. While this process does not alter the final tally, it prevents voters from decrypting their original vote. This step mitigates vote selling or coercion, as voters are no longer able to prove their choice to third parties. Finally, sequencers submit the reencrypted votes along with their verification and reencryption proofs to the process management smart contract. The smart contract verifies all the proofs provided by the sequencers to check that they complied with the protocol.

4. *Tally decryption.* At the end of the voting period, the smart contract ceases to accept new state updates. Then, a threshold number of key wardens cooperate to reconstruct a partial decryption key that allows to decrypt the final tally. That is, instead of reconstructing the election's private key, each key warden publishes a partial decryption together with a proof that the contribution is correct. This way, the individual votes remain encrypted throughout the voting period, and only the final aggregated result is decrypted at the end of the election.

5. *Election finalization.* The results and a proof of correctness are submitted to the results verification smart contract. The contract verifies the proof, ensuring that the tally has been derived faithfully from the final state. Once this verification succeeds, the results and the latest state root remain available on-chain, providing an immutable record of the election outcome that can be independently audited by anyone.

4 Voting protocol

This section outlines the DAVINCI voting protocol. First, we introduce the parties involved in the process in Section 4.1. In Section 4.2 we present the Ethereum smart contracts that rule the DAVINCI protocol. Then, in Section 4.3, we describe the census data structures and in Section 4.4 the state Merkle tree. In Section 4.5, we present the ZK circuits used to enforce vote validity, authentication, aggregation, state transitions, and results computation. Finally, in Section 4.6, we detail the full protocol flow step by step, from the voting setup to the results validation and process finalization.

4.1 Parties involved

An election involves four types of participants: organizer, key wardens, voters, and sequencers.

Organizer. The organizer is the entity responsible for defining and setting up the election. Its key responsibilities include defining the voting parameters and gathering the census data. The organizer ensures that the election is structured correctly but does not participate in vote collection or tallying.

Key wardens. Key wardens are a set of decentralized parties that collaboratively generate a public encryption key that voters use to encrypt their votes. The corresponding secret key is secret-shared among them so that a threshold of key wardens can later collaborate to decrypt the final tally.

Voters. Voters are the participants that belong the census and are allowed to cast their votes in the election.

Sequencers. Sequencers are a set of parties that during the voting period receive and verify encrypted ballots from voters, reencrypt votes, and update the shared public state accordingly.

4.2 Smart contracts

The DAVINCI protocol operates on an Ethereum-compatible network, which serves as the source of truth for the election. By leveraging an Ethereum virtual machine (EVM) blockchain, all election transitions become immutable and publicly verifiable. To coordinate and validate each phase of the process, DAVINCI deploys a set of smart contracts that enforce protocol rules, verify ZK proofs, and maintain on-chain commitments such as state roots, encryption keys, and final results. Together, these contracts provide a trustless execution environment that guarantees the correct and transparent progression of the election without relying on any centralized authority.

Organization registry. The OrganizationRegistry smart contract is used to create and manage organizations within the system. Each organization is identified by the Ethereum address of its creator and is associated with a name and a URL pointing to its metadata. At this stage, the contract serves only as a registration mechanism, but it lays the foundation for future extensions, such as managing multiple elections under the same organization.

Census management. This contract is only used by the on-chain dynamic census model (see Section 4.3), since the off-chain and credential service provider (CSP) models store the census commitment directly as a parameter and require no such contract. This contract maintains the census as an on-chain IMT and exposes (i) the current census root, (ii) the total voting power recorded for a given root, and (iii) the validity of a root, returning the last block at which it was valid.

Key management. The KeyManagement smart contract coordinates the DKG process among the key wardens. It records their contributions, verifies proofs of correct participation, and ensures that a valid encryption public key is produced once the DKG round is complete.

Election registry. The ElectionRegistry smart contract manages the lifecycle of each election, from its creation to the registration of final results. It maintains the election's current state and ensures that all transitions follow the protocol.

State transition verifier. The StateTransitionVerifier is responsible for validating the proofs that attest to correct updates of the election state. It stores the verification key corresponding to the state transition circuit (see Section 4.5.4) and checks that each submitted proof correctly links the previous and new state roots. Only transitions verified through this contract are accepted as valid updates by the process registry.

Results verifier. The ResultsVerifier contract validates the proof of correct tally computation. It contains the verification key for the results circuit (see Section 4.5.5) and ensures that the final results submitted by the organizer match the commitments from the final state. Once verified, the results and their proof are permanently recorded on-chain, providing a tamper-proof reference for the election outcome.

4.3 Census

The census defines the set of eligible voters and their associated voting weights. At its core, a census is simply a dataset that, for each voter, contains a unique identifier (typically an Ethereum address), a sequential index, a voting weight, and a means to produce a membership proof that can be verified inside a ZK circuit. DAVINCI supports two mechanisms for generating such proofs:

- *Credential-based membership proofs.* Eligibility is certified by a digital signature issued by a CSP. The CSP assigns a unique index (`idx`) to each voter and includes it in the signed credential alongside the voter's identifier (`address`), their weight (`weight`), and the process identifier (`processID`).
- *Merkle-tree-based membership proofs.* The census is organized as an IMT (see Appendix A.3) and denoted by $\mathsf{MT}^{\texttt{census}}$. Each voter is assigned a unique sequential index (`idx`) at the time of registration, and their identifier (`address`) and weight (`weight`) are stored as a leaf at position `idx` in the tree. A membership proof (`censusProof`) for voter i is a Merkle path attesting that the tuple $(\texttt{address}_i, \texttt{weight}_i)$ is stored at position $\texttt{idx}_i$ under the committed census tree root. Sequencers supply these Merkle proofs to attest voters' membership to the census and the index `idx` is used to derive the voter's reserved storage slot in the state tree (see Section 4.4).

To support the different census models, the ElectionRegistry smart contract stores the following parameters:

- `censusOrigin`: the deployment model (off-chain static, off-chain dynamic, on-chain dynamic, or CSP).
- `censusURI`: a uniform resource identifier (URI) pointing to an external reference (URL, GraphQL endpoint, CDN path, IPFS/IPNS, etc.) from which sequencers can download or query the census data.
- `censusRoot`: the on-chain commitment used for membership verification. Its interpretation depends on the `censusOrigin`. In the CSP model, this parameter corresponds to the hash of the CSP's public key, and in the case of Merkle trees, it corresponds to a Merkle root (off-chain static/dynamic) or a census contract address (on-chain dynamic).

Organizers are free to construct the census from various data sources, such as private membership registries, self-sovereign identity credentials, or Ethereum-based tokens (e.g. ERC-20 or NFTs) whose balances define eligibility. This flexibility supports a broad range of use cases and governance scenarios. The supported census models are summarized in Table 1 and described in detail below.

Credential service provider. Voter eligibility is certified by a digital signature issued by a CSP. A voter obtains a signed credential from the CSP, and sequencers verify the signature inside the ZK circuits. The credential is issued over the tuple (`processID`, `idx`, `address`, `weight`) where `idx` is a unique index assigned by the CSP to each voter. This index will be used by the state transition circuit to derive the voter's reserved storage slot in the state tree (see Section 4.4). In this case, `censusRoot` stores the hash of the CSP's public key, and `censusURI` specifies the endpoint where voters obtain their credential. This model is suitable when eligibility is managed externally through attested identities rather than by maintaining a Merkle-tree dataset.

Off-chain static census. The organizer constructs a fixed census prior to the election. The Merkle tree is built locally and the resulting root is stored in `censusRoot`, and `censusURI` specifies the endpoint from which sequencers can retrieve the full tree. Since the census is static, no updates occur during the voting period.

Off-chain dynamic census. The Merkle tree remains off-chain, but the organizer may append new voters during the voting period (never deleting voters or modifying weights, as this would enable double voting). Each new insertion produces a new Merkle root, which the organizer must publish to the ElectionRegistry contract by updating the `censusRoot` parameter. As in the static model, `censusURI` provides the endpoint from which sequencers can download the current version of the tree. Sequencers verify membership proofs against the most recently published root.

On-chain dynamic census. In this configuration, the census is managed by a dedicated on-chain contract (CensusManagement) that supports adding new members during the voting period. The address of this contract is stored in `censusRoot`, while `censusURI` points to an external representation of the tree. The CensusManagement contract maintains a history of valid Merkle roots. Sequencers may therefore generate membership proofs using older roots, and during state transitions the ElectionRegistry contract queries the CensusManagement contract to verify that the root used in the proof is still valid. As in all dynamic models, only additions to the census are permitted during the voting period.

`censusOrigin`		`censusRoot`	`censusURI`	Membership proof
Credential service provider (CSP)		Hash of the CSP's public key	Endpoint to obtain an eligibility credential	Digital signature verification
Off-chain	Static	Fixed Merkle root	Endpoint to obtain the census tree	Merkle proof verified against the fixed Merkle root
	Dynamic (add-only)	Latest Merkle root	Endpoint to obtain the latest version of the tree	Merkle proof verified against the latest Merkle root
On-chain	Dynamic (add-only)	CensusManagement contract address	Endpoint to obtain the latest version of the tree	Merkle proof verified against most recent Merkle roots

Table 1: Summary of census configurations supported by the DAVINCI protocol.

4.4 State tree

The state Merkle tree, denoted by $\mathsf{MT}^{\texttt{state}}$, represents the evolving global state of an election in DAVINCI. It is implemented as a SMT of fixed depth $D = 64$, as described in Appendix A.3. The state tree acts as a cryptographic accumulator that compactly commits to all relevant election data, including configuration parameters, vote identifiers, and encrypted ballots. By organizing the election state in a single SMT, DAVINCI supports efficient membership and update proofs that can be verified inside ZK circuits, enabling succinct and verifiable state transitions.

The root of the state tree, denoted `stateRoot`, is committed on-chain and serves as the authoritative reference to the current election state. Each state transition performed by a sequencer updates the tree and produces a new root. The correctness of this update, namely, that it follows the protocol rules and correctly

incorporates new votes, is attested by a ZK-SNARK proof, which is verified by the StateTransitionVerifier smart contract before the new root is accepted.

Namespaced key space. The state tree operates over keys in the range $[0, 2^D - 1] \subset \mathbb{F}_p$ and uses a SNARK-friendly hash function (Poseidon, see Appendix A.2) to enable efficient in-circuit verification. To prevent collisions between different categories of data stored in the state, the 64-bit key space is partitioned into three disjoint numeric namespaces using a fixed thershold N. We define $N = 2^{D-1}$, which we use to split the address space into a lower region for configuration parameters and encrypted ballot storage, and an upper region strictly reserved for vote identifiers. This strict separation ensures that configuration entries, vote identifiers, and ballots never collide.

Configuration namespace. The lowest indices of the tree are reserved for a small set of process parameters. Most are configuration values fixed at election setup and never modified thereafter (`processID`, `ballotMode`, `encryptionKey`, `censusOrigin`); the remaining entry is a mutable accumulator (`encryptedResults`) updated on every state transition. In all cases these keys occupy deterministic, well-known positions and use a negligible portion of the address space. The default entries are:

- `0x0`: the election identifier (`processID`). [4]
- `0x2`: the ballot mode configuration, which is the set of rules for validating votes (`ballotMode`).
- `0x3`: the encryption public key used to encrypt the ballots (`encryptionKey`).
- `0x4`: the running encrypted results (`encryptedResults`).
- `0x6`: the census origin (`censusOrigin`).

Let `configMax` denote the largest index reserved for configuration parameters (by default, `configMax` $= 15$).

Encrypted ballot namespace. The region immediately following the configuration parameters, the interval $[\texttt{configMax} + 1, N - 1]$, is reserved for storing encrypted ballots. Ballot storage locations are deterministic and derived from the voter's position in the census. Let `idx` denote the voter's census index (see Section 4.3), proven via a census membership proof. The storage path for the encrypted ballot is computed as

$$\text{encPath} = \texttt{configMax} + \texttt{idx} \cdot 2^{16} + (\texttt{address} \bmod 2^{16}).$$

This construction assigns each eligible voter a unique, reserved leaf in the state tree. Because the index `idx` is authenticated by the census, a voter can only write to their own slot. This enables efficient *last-vote-wins* semantics: if a voter overwrites their ballot, the new ciphertext replaces the previous one at the same path, while the tally is updated by subtracting the old contribution and adding the new one.

Vote identifier namespace. The upper portion of the tree, $[N, 2^D - 1]$ is reserved for *vote identifiers* (`voteID`), which allows voters to verify that their vote has been included in the election state, without revealing or linking to the corresponding encrypted ballot. Vote identifiers are derived by hashing the process identifier, the voter's address, and fresh randomness, and then mapping the result into the allowed range by setting the most significant bit to 1:

$$\texttt{voteID} = N + \big(\mathsf{Poseidon}(\texttt{processID}, \texttt{address}, k) \bmod N\big).$$

The StateTransitionCircuit recomputes this derivation and enforces that $N \leq \texttt{voteID} < 2^D$, guaranteeing that vote identifiers cannot overlap with configuration entries or ballot storage.

Unlike classical nullifiers, vote identifiers are not used to prevent duplicate voting directly. Instead, correctness is enforced at the circuit level: the state transition circuit guarantees that a vote identifier can only be inserted if the corresponding leaf in the state tree is empty. This emptiness check is enforced inside the ZK circuit. Since the vote identifier depends on a fresh random value k, if a collision occurs (i.e., the computed

[4] The indices `0x1` and `0x5` are currently unused: `0x1` is reserved for backward compatibility (historically used for alternative census structures), and `0x5` is reserved for future use (an earlier design stored a separate subtractive results accumulator there; the current design keeps a single net results accumulator at `0x4`).

leaf is already occupied), the circuit will reject the transition. In that case, the voter must resample k and recompute a new vote identifier until an unused leaf is obtained. As a result, collisions do not compromise correctness or liveness, but merely require retrying the identifier derivation.

Note that the maximum number of vote identifiers that can be stored is bounded by the size of this namespace (2^{D-1}), which upper-bounds the total number of votes and overwrites that can be processed throughout the election. This capacity is orders of magnitude larger than any realistic election workload: historical elections peak at $\approx 10^9$ votes [34], ensuring that identifier exhaustion is practically impossible and that collisions are exceptionally rare [5].

Together, the state tree provides a compact and tamper-proof commitment to the current status of the election. At the end of the election, the final state root together with the on-chain verification of all transitions serves as a complete cryptographic record of the election, encapsulating both the tally and the integrity of the entire voting procedure.

4.5 Circuits

At the core of DAVINCI lie a set of arithmetic circuits that enforce the correctness of every step of the election. Each circuit corresponds to a distinct task and, taken together, they guarantee that all protocol rules are satisfied without revealing any private information. Specifically, the *ballot circuit* (Section 4.5.1) ensures that encrypted votes are valid and well-formed, the *verifier circuit* (Section 4.5.2) verifies the voter's proof; the *aggregation circuit* (Section 4.5.3) combines multiple authenticated votes into a single proof; the *state transition circuit* (Section 4.5.4) updates the global state root and produces the ZK-SNARK proof that is verified on-chain; and the *results circuit* (Section 4.5.5) proves that the election results were computed correctly. The flow of these circuits and their interactions is shown in Fig. 2.

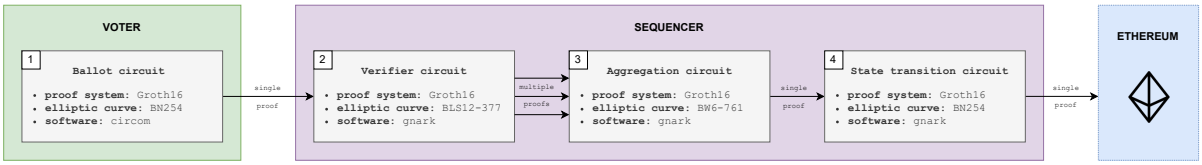

Fig. 2: Flow of the voter and sequencers circuits.

Remark 1. For clarity, the circuits are described in this section with their full set of public inputs explicitly listed. In the actual implementation, however, we apply an optimization: rather than exposing all public inputs PI individually, we compute a single commitment $h = \mathsf{Poseidon}(\mathsf{PI})$ and use h as the only public input. The proof verifier then checks that $\mathsf{Poseidon}(\mathsf{PI}) = h$, ensuring that the original public inputs are consistent while reducing both proof size and verification cost. Note that this optimization does not alter the semantics of the circuits but significantly reduces verifier complexity and on-chain verification costs. Other implementation aspects such as the exact number of constraints and the proving frameworks used are deferred to Section 6.2.

4.5.1 Ballot circuit

The ballot circuit ($\mathsf{BallotCircuit}$), illustrated in Fig. 3, is generated locally by the voter at the time of casting a ballot. Its purpose is twofold: first, to prove that the encrypted ballot is valid and complies with the protocol rules defined by the organizer; and second, to prove that the vote identifier (`voteID`) has been correctly

[5]Based on the birthday paradox, the probability of at least one collision occurring is given by $P = 1 - e^{-n^2/2d}$, where n is the number of vote identifiers and $d = 2^{D-1}$. For a large-scale national election of 10^8 votes emitted, this probability is vanishingly small ($\approx 0.054\%$), and for an extreme workload of 10^9 votes, the probability of a single collision is only $\approx 5.4\%$, meaning that the need to resample k remains highly improbable.

derived. The correctness of this circuit is attested by a ZK proof generated using the ZK-SNARK protocol described in Appendix A.8, instantiated over the BN254 curve. We call the resulting proof *ballot proof* and denote it as `ballotProof`. We detail below the inputs and constraints of the ballot circuit.

Public inputs

- public `processID`: election identifier.
- public `ballotMode`: ballot protocol configuration.
- public `encryptionKey`: encryption public key.
- public `address`: voter's address.
- public `weight`: voter's weight.
- public `encryptedBallot`: encrypted ballot.
- public `voteID`: vote's unique identifier.

Private inputs

- private `ballot`: voter's voting choice.
- private k: random scalar.

Constraints

- *Vote encryption.* Correct encryption of the ballot:

$$\texttt{encryptedBallot} = \mathsf{Enc}_{\texttt{encryptionKey}}(\texttt{ballot}; k).$$

- *Protocol rules compliance.* The voter's ballot meets the protocol rules (see Section 5):

$$\text{protocolRulesCompliance}(\texttt{ballotMode}, \texttt{weight}, \texttt{ballot}) = \texttt{true}.$$

- *Vote identifier.* The vote's identifier is correctly computed (by default, $N = 2^{63}$, see Section 4.4):

$$\texttt{voteID} = N + \big(\mathsf{Poseidon}(\texttt{processID}, \texttt{address}, k) \bmod N\big).$$

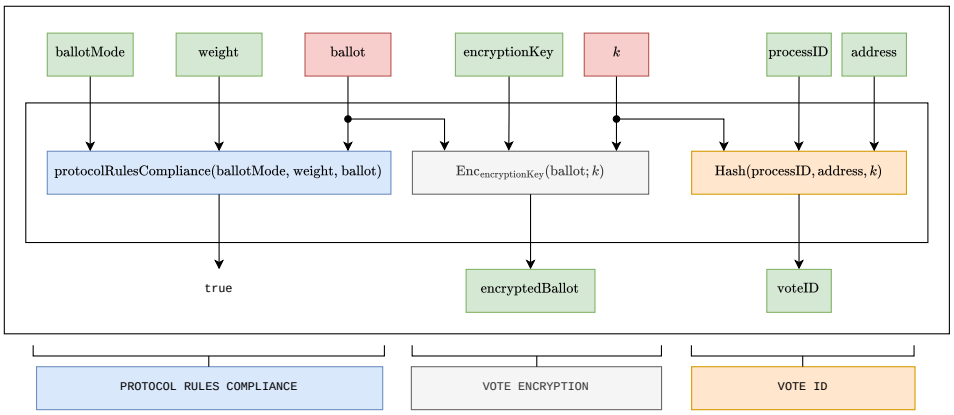

Fig. 3: Ballot circuit. All public values are framed in green.

4.5.2 Verifier circuit

The verifier circuit (VerifierCircuit), illustrated in Fig. 4, is generated by the sequencer when processing a vote. This circuit verifies the ballot's proof generated by the voter and the correctness of their digital signature. The correctness of this circuit is attested by a ZK proof generated using the ZK-SNARK protocol described in Section A.8, instantiated over the BLS12-377 curve. We call the resulting proof the *authentication proof* and denote it as `authenticationProof`. We detail below the inputs and constraints of this circuit.

Public inputs

- public `processID`: election identifier.
- public `ballotMode`: ballot protocol configuration.
- public `encryptionKey`: encryption public key.
- public `encryptedBallot`: encrypted ballot.
- public `voteID`: vote's unique identifier.
- public `address`: voter's address.
- public `weight`: voter's weight.

Private inputs

- private pk: voter's public key.
- private `signature`: voter's signature.
- private `ballotProof`: voter's ballot proof.

Constraints

- *Authentication + non-malleability.* The voter submitted and signed their vote.
  - The signature of the vote's identifier is valid for the given public key:

$$\mathsf{S.Verify}_{\mathsf{pk}}(\texttt{voteID}, \texttt{signature}) = \texttt{true}.$$

  - The address is correctly derived from the user's public key:

$$\texttt{address} = \mathsf{Keccak}(\mathsf{pk}).$$

- *Voter's proof verification.* The voter's submitted proof is valid for the inputs provided.
  - Set public inputs:

$$\mathsf{PI} = (\texttt{processID}, \texttt{ballotMode}, \texttt{encryptionKey}, \texttt{encryptedBallot}, \texttt{voteID}, \texttt{weight}, \texttt{address}).$$

  - Verify proof:

$$\mathsf{P.Verify}(\texttt{ballotProof}, \mathsf{PI}) = \texttt{true}.$$

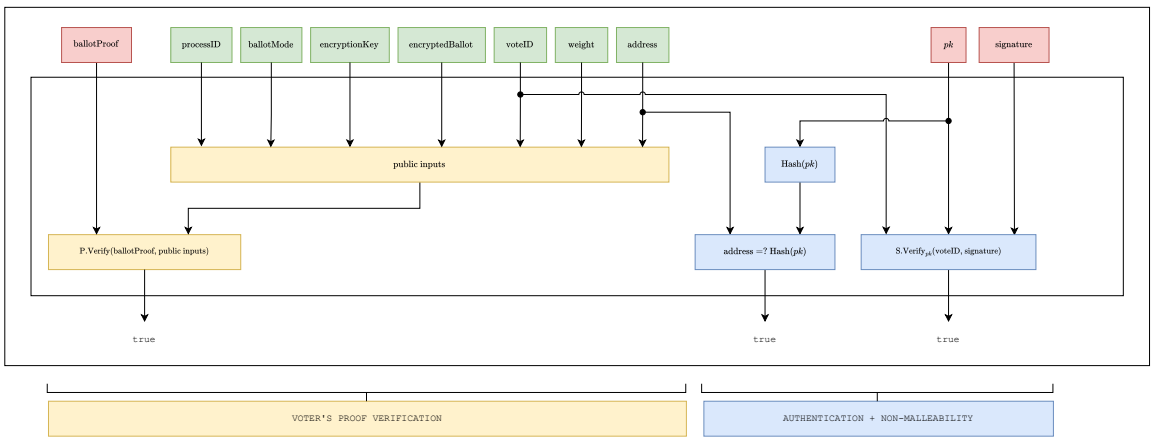

Fig. 4: Authentication circuit. All public values are framed in green.

4.5.3 Aggregation circuit

The aggregation circuit (AggregationCircuit), illustrated in Fig. 5, is generated by the sequencer to combine multiple authenticated votes into a single proof. Its purpose is to reduce verification overhead by recursively aggregating individual authentication proofs, while ensuring that all aggregated votes belong to the same election. The batch size (`batchSize`), i.e., the maximum number of proofs aggregated in a single execution,

is a fixed parameter (currently set to 60). If fewer proofs are available, the sequencer pads the batch with dummy proofs so that the circuit always operates on a fixed-size input. The correctness of this circuit is attested by a ZK proof generated using the ZK-SNARK protocol described in Appendix A.8, instantiated over the BW6-761 curve. We call the resulting proof the *aggregation proof* and denote it as `aggregationProof`. We detail below the inputs and constraints of the aggregation circuit.

Public inputs

- public $\{\texttt{encryptedBallot}_i\}_{i=1}^{\texttt{batchSize}}$: set of voters' encrypted ballots.
- public $\{\texttt{voteID}_i\}_{i=1}^{\texttt{batchSize}}$: set of vote identifiers.
- public $\{\texttt{address}_i\}_{i=1}^{\texttt{batchSize}}$: set of voters' addresses.
- public `processID`: election identifier.
- public `ballotMode`: ballot protocol configuration.
- public `encryptionKey`: encryption public key.

Private inputs

- private $\{\texttt{authenticationProof}_i\}_{i=1}^{\texttt{batchSize}}$: set of authentication proofs.

Constraints

- *Votes aggregation.* The accumulated authentication proofs are valid. For every $i \in \{1, \ldots, \texttt{batchSize}\}$:
  - Set public inputs:

$$\mathsf{PI} = (\texttt{encryptedBallot}_i, \texttt{voteID}_i, \texttt{address}_i, \texttt{weight}_i, \texttt{processID}, \texttt{ballotMode}, \texttt{encryptionKey}).$$

  - Verify proof:

$$\mathsf{P.Verify}(\texttt{authenticationProof}_i, \mathsf{PI}) = \texttt{true}.$$

- *Shared public inputs.* All authentication proofs are verified using the same public inputs `processID`, `ballotMode`, and `encryptionKey`.

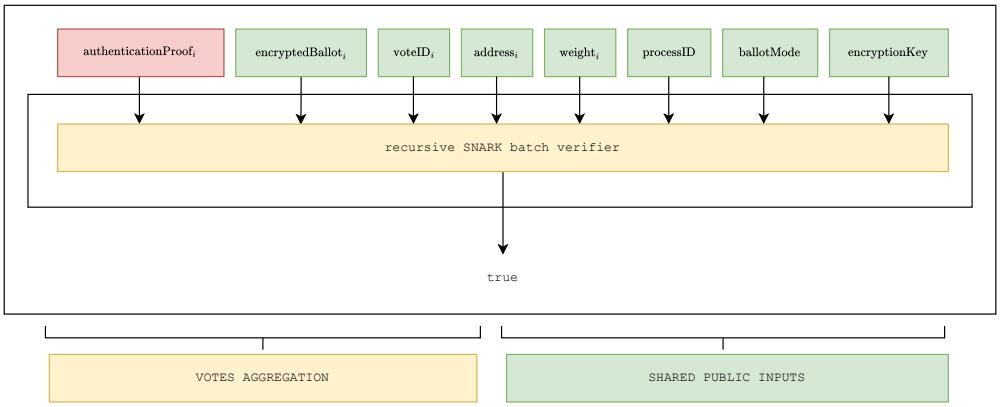

Fig. 5: Aggregation circuit. All public values are framed in green.

4.5.4 State transition circuit

The state transition circuit (StateTransitionCircuit), illustrated in Fig. 6, is generated by the sequencer to update the global state of the election. Its purpose is to verify that a batch of aggregated votes has been correctly incorporated into the state Merkle tree, that overwrites and vote identifiers are handled consistently, that the accumulators of encrypted results are updated accordingly, that each included vote corresponds to an eligible voter included in the census, and that the state transition data used by the sequencer matches the data made available through Ethereum data blobs. The correctness of this circuit is attested by a ZK proof

generated using the ZK-SNARK protocol described in Appendix A.8, instantiated over the BN254 curve, which is supported by Ethereum's native precompiles for proof verification. We call the resulting proof the *state proof* and denote it as `stateProof`. It is submitted on-chain and verified by the process management smart contract, ensuring that every accepted state transition complies with the protocol rules.

To formalize the setting, let the sequencer collect a batch of N votes ($N \leq 60$), of which n are new vote submissions and $m = N - n$ are overwrites of previously cast ballots. Without loss of generality, we assume that the batch is ordered such that the first n votes correspond to new submissions, and the remaining m votes correspond to overwrites. In the following, we describe the inputs and constraints of the state transition circuit in detail.

Public inputs

- public $\texttt{censusRoot}$: Merkle root of the census tree.
- public $\texttt{stateRoot}^{\texttt{old}}$: current Merkle root of the state tree.
- public $\texttt{stateRoot}^{\texttt{new}}$: new tree root after all state transitions are applied.
- public $\texttt{numOverwrites}^{\texttt{old}}$: current number of overwrites.
- public $\texttt{numOverwrites}^{\texttt{new}}$: number of overwrites after all state transitions are applied.
- public $\texttt{numVotes}^{\texttt{old}}$: current number of votes.
- public $\texttt{numVotes}^{\texttt{new}}$: total number of votes after all state transitions are applied.
- public C: commitment to the data blob.

Private inputs

- private $\texttt{processID}$: election identifier.
- private $\texttt{censusOrigin}$: census origin parameter.
- private $\texttt{ballotMode}$: ballot protocol configuration.
- private $\texttt{encryptionKey}$: encryption public key.
- private $\texttt{encryptedResults}^{\texttt{old}}$: current accumulator of votes (before transitions are applied).

- private $\{\texttt{aggregationProof}_i\}_{i=1}^{N}$: set of aggregation proofs.
- private $\{\texttt{censusProof}_i\}_{i=1}^{N}$: census membership proofs.
- private $\{\texttt{voteID}_i\}_{i=1}^{N}$: set of vote's identifiers.
- private $\{\texttt{encryptedBallot}_i\}_{i=1}^{N}$: set of voters' encrypted ballots.
- private $\{\texttt{encryptedBallot}_i^{\texttt{old}}\}_{i=n+1}^{N}$: set of all old encrypted ballots that will be overwritten.
- private $\{\texttt{address}_i\}_{i=1}^{N}$: list of voters' addresses.
- private $\{\texttt{weight}_i\}_{i=1}^{N}$: list of voters' weights.

- private $\texttt{inclusionProof}^{\texttt{processID}}$: proof that $\texttt{processID}$ is stored in leaf $\texttt{0x0}$ of the state tree.
- private $\texttt{inclusionProof}^{\texttt{ballotMode}}$: proof that $\texttt{ballotMode}$ is stored in leaf $\texttt{0x2}$ of the state tree.
- private $\texttt{inclusionProof}^{\texttt{encryptionKey}}$: proof that $\texttt{encryptionKey}$ is stored in leaf $\texttt{0x3}$ of the state tree.
- private $\texttt{inclusionProof}^{\texttt{encryptedResults}}$: proof that $\texttt{encryptedResults}^{\texttt{old}}$ is in leaf $\texttt{0x4}$ of the state tree.
- private $\texttt{inclusionProof}^{\texttt{censusOrigin}}$: proof that $\texttt{censusOrigin}$ is stored in leaf $\texttt{0x6}$ of the tree.

- private $\{\texttt{nonInclusionProof}_i^{\texttt{encryptedBallot}}\}_{i=1}^{n}$: proof that the leaf corresponding to a voter's $\texttt{address}_i$ that submits a new vote is empty.
- private $\{\texttt{inclusionProof}_i^{\texttt{encryptedBallot}}\}_{i=n+1}^{N}$: proof that the vote $\texttt{encryptedBallot}_i^{\texttt{old}}$ is stored in the leaf corresponding to a voter's $\texttt{address}$ that submits the vote overwrite $\texttt{encryptedBallot}_i$.
- private $\{\texttt{nonInclusionProof}_i^{\texttt{voteID}}\}_{i=1}^{N}$: non-membership proofs of vote identifiers.

- private k: random scalar.
- private openingProof: KZG opening proof.

Constraints

- *Aggregation verification.* The accumulated authentication proofs are valid. For every $i \in \{1, \ldots, N\}$:
  - Set public inputs:

$$\mathsf{PI} = (\texttt{weight}_i, \texttt{address}_i, \texttt{encryptedBallot}_i, \texttt{voteID}_i, \texttt{ballotMode}, \texttt{encryptionKey}, \texttt{processID}).$$

- Verify proof:

$$\mathsf{P.Verify}(\mathtt{aggregationProof}_i, \mathsf{PI}) = \mathtt{true}.$$

- *Census membership.* Each processed vote must correspond to an eligible voter, according to the census configuration specified by the parameter `censusOrigin`.
  - If `censusOrigin` = CSP, then verify a signature issued by the CSP. Concretely, for all $i \in \{1, \dots, N\}$:

$$\mathsf{S.Verify}_{\mathtt{censusRoot}}\big((\mathtt{processID}, \mathtt{idx}_i, \mathtt{address}_i, \mathtt{weight}_i), \mathtt{censusProof}_i\big) = \mathtt{true},$$

    where `censusRoot` contains the CSP public key and $\mathtt{censusProof}_i$ is the eligibility signature attesting that voter i belongs to the census with the given weight.
  - Otherwise, if the census is represented by an IMT, the circuit verifies a Merkle membership proof showing that the pair $(\mathtt{address}_i, \mathtt{weight}_i)$ is included in the committed census. For all $i \in \{1, \dots, N\}$:

$$\mathsf{MT}^{\mathtt{census}}.\mathsf{Verify}\big((\mathtt{address}_i, \mathtt{weight}_i), \mathtt{idx}_i, \mathtt{censusProof}_i, \mathtt{censusRoot}\big) = \mathtt{true}.$$

- *Transition verification.* Verify that the initial state and the state transition updates are applied correctly.
  - Verify that the inputs `processID`, `ballotMode`, `encryptionKey`, $\mathtt{encryptedResults}^{\mathtt{old}}$, and `censusOrigin` correspond to the content of the first state tree leaves.

$$\mathsf{MT}^{\mathtt{state}}.\mathsf{Verify}(\mathtt{processID}, \mathtt{0x0}, \mathtt{inclusionProof}^{\mathtt{processID}}, \mathtt{stateRoot}^{\mathtt{old}}).$$
$$\mathsf{MT}^{\mathtt{state}}.\mathsf{Verify}(\mathtt{ballotMode}, \mathtt{0x2}, \mathtt{inclusionProof}^{\mathtt{ballotMode}}, \mathtt{stateRoot}^{\mathtt{old}}).$$
$$\mathsf{MT}^{\mathtt{state}}.\mathsf{Verify}(\mathtt{encryptionKey}, \mathtt{0x3}, \mathtt{inclusionProof}^{\mathtt{encryptionKey}}, \mathtt{stateRoot}^{\mathtt{old}}).$$
$$\mathsf{MT}^{\mathtt{state}}.\mathsf{Verify}(\mathtt{encryptedResults}^{\mathtt{old}}, \mathtt{0x4}, \mathtt{inclusionProof}^{\mathtt{encryptedResults}}, \mathtt{stateRoot}^{\mathtt{old}}).$$
$$\mathsf{MT}^{\mathtt{state}}.\mathsf{Verify}(\mathtt{censusOrigin}, \mathtt{0x6}, \mathtt{inclusionProof}^{\mathtt{censusOrigin}}, \mathtt{stateRoot}^{\mathtt{old}}).$$

  - Verify that none of the $\mathtt{voteID}_i$ for all $i \in \{1, \dots, N\}$ are initially part of the tree:

$$\mathsf{MT}^{\mathtt{state}}.\mathsf{Verify}(\mathtt{voteID}_i, \mathtt{nonInclusionProof}_i^{\mathtt{voteID}}, \mathtt{stateRoot}^{\mathtt{old}}).$$

  - Ensure that new votes are not part of the tree yet. For $i = 1, \dots, n$:

$$\mathsf{MT}^{\mathtt{state}}.\mathsf{Verify}(\mathtt{address}_i, \mathtt{nonInclusionProof}_i^{\mathtt{encryptedBallot}}, \mathtt{stateRoot}^{\mathtt{old}}).$$

  - Ensure that overwrites do correspond to votes that were already on the tree. For $i = 1, \dots, m$:

$$\mathsf{MT}^{\mathtt{state}}.\mathsf{Verify}(\mathtt{encryptedBallot}_i^{\mathtt{old}}, \mathtt{address}_i, \mathtt{inclusionProof}_i^{\mathtt{encryptedBallot}}, \mathtt{stateRoot}^{\mathtt{old}}).$$

  - State transitions.
    - For every new vote $i = 1, \dots, n$:
      1. After every use, refresh reencryption randomness: $k = \mathsf{Poseidon}(k)$.
      2. Reencrypt the ballot: $\mathtt{encryptedBallot}_i = \mathsf{ReEnc}_{\mathtt{encryptionKey}}(\mathtt{encryptedBallot}_i, k)$.
      3. Add reencrypted ballot to the tree: $\mathsf{MT}^{\mathtt{state}}.\mathsf{Insert}(\mathtt{encryptedBallot}_i, \mathtt{address}_i)$.
      4. Add vote identifier to the tree: $\mathsf{MT}^{\mathtt{state}}.\mathsf{Insert}(\mathtt{voteID}_i)$.
      5. Increase votes counter: $\mathtt{numVotes} = \mathtt{numVotes} + 1$.
      6. Aggregate the vote to the results accumulator: $\mathtt{encryptedResults} = \mathtt{encryptedResults} + \mathtt{encryptedBallot}_i$.
    - For every overwritten vote $i = n + 1, \dots, N$:
      1. After every use, refresh reencryption randomness: $k = \mathsf{Poseidon}(k)$.
      2. Reencrypt the ballot: $\mathtt{encryptedBallot}_i = \mathsf{ReEnc}_{\mathtt{encryptionKey}}(\mathtt{encryptedBallot}_i, k)$.
      3. Overwrite the encrypted ballot: $\mathsf{MT}^{\mathtt{state}}.\mathsf{Update}(\mathtt{encryptedBallot}_i, \mathtt{address}_i)$.
      4. Add vote identifier: $\mathsf{MT}^{\mathtt{state}}.\mathsf{Insert}(\mathtt{voteID}_i)$.
      5. Increase votes counter: $\mathtt{numVotes} = \mathtt{numVotes} + 1$.
      6. Increase overwrites counter: $\mathtt{numOverwrites} = \mathtt{numOverwrites} + 1$.
      7. Substract the old ballot from the results accumulator and aggregate the new one: $\mathtt{encryptedResults} = \mathtt{encryptedResults} - \mathtt{encryptedBallot}_i^{\mathtt{old}} + \mathtt{encryptedBallot}_i$.

- After all previous state transitions are done, recompute new tree root:

$$\texttt{stateRoot}^{\texttt{new}} = \mathsf{MT}^{\texttt{state}}.\mathsf{Root}().$$

- Reencrypt a set of old leafs from the tree: first, very their Merkle tree inclusion proofs, then reencrypt them, and finally update each leaf's content.

- *Data blob.*
  - From all leaves that changed after the state transitions in the $\mathsf{MT}^{\texttt{state}}$ tree, construct blob B.
  - Derive polynomial $p(x)$ from B.
  - Compute evaluation point $z = \mathsf{Poseidon}(\texttt{processID}, C, \mathsf{openingProof})$.
  - Compute evaluation $y = p(z)$.
  - Verify $\mathsf{C.Verify}(z, y, C, \mathsf{openingProof}) = \texttt{true}$.

Remark 2. Splitting the sequencer computation into three circuits provides an important efficiency benefit. If the global `stateRoot` changes while a sequencer is preparing a proof, only the last circuit must be re-run to reflect the updated state, avoiding the need to recompute all previous proofs.

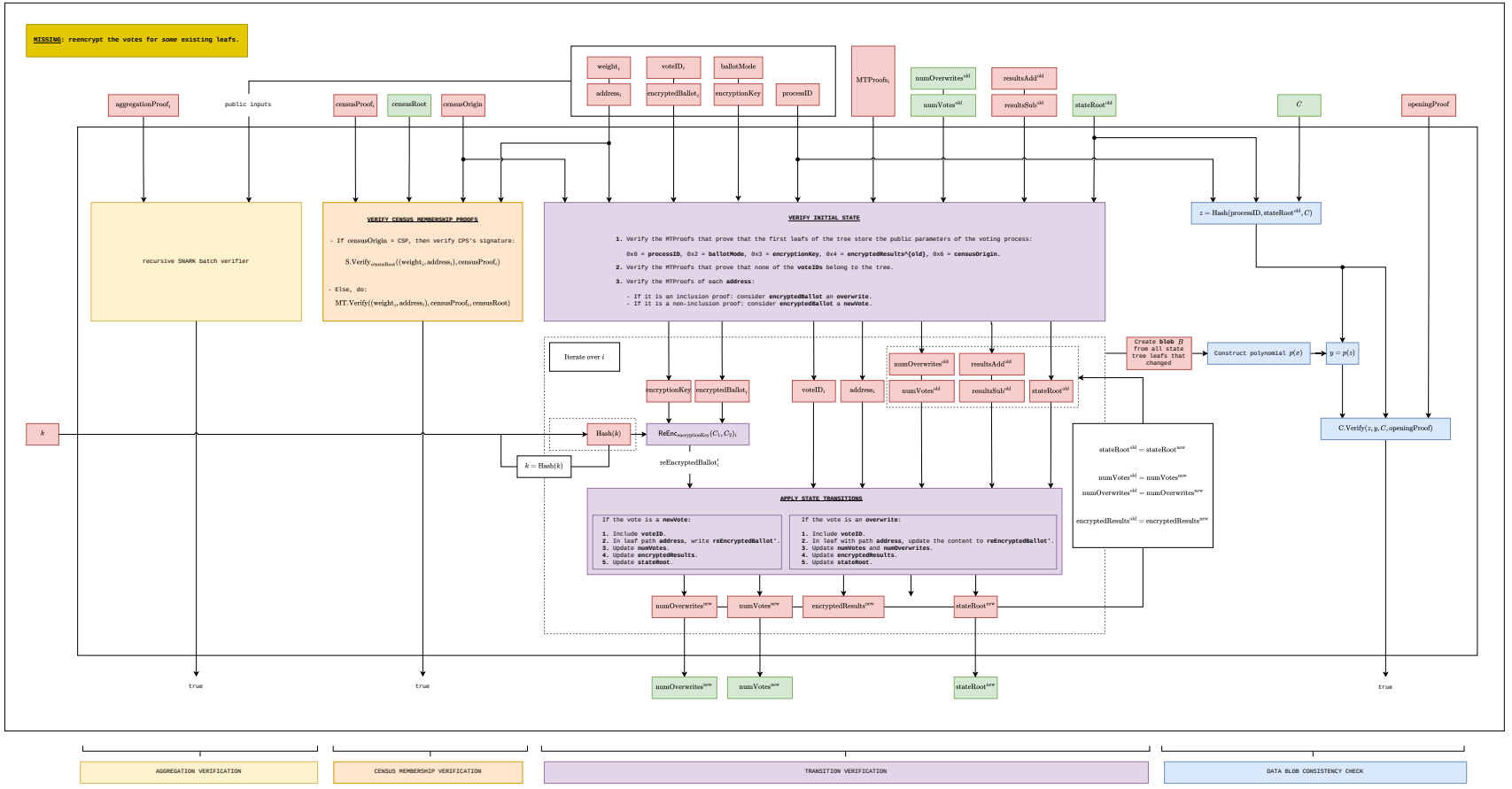

Fig. 6: State transition circuit. All public values are framed in green.

4.5.5 Results circuit

The results circuit (ResultsCircuit) is generated at the end of the election, by the organizer or a sequencer, to prove that the published tally was correctly obtained from the encrypted results committed in the final state. Crucially, the circuit does not take the decryption key as input, nor does it decrypt inside the circuit: instead, for each ballot field it verifies a proof of correct decryption against the encryption public key that is committed in the state tree. The correctness of this circuit is attested by a ZK proof generated using the ZK-SNARK protocol described in Appendix A.8, instantiated over the BN254 curve, so that it can be verified on-chain by the ResultsVerifier contract using Ethereum's native precompiles. We call the resulting proof the *results proof* and denote it as `resultsProof`. We detail below the inputs and constraints of the results circuit.

Public inputs

- public `stateRoot`: final root of the state tree.
- public $\{\text{results}_j\}_{j=1}^{n_f}$: decrypted tally, one entry per ballot field.

Private inputs

- private `encryptedResults`: encrypted results accumulator stored in the state (n_f ElGamal ciphertexts).
- private `encryptionKey`: encryption public key.
- private $\texttt{inclusionProof}^{\texttt{encryptedResults}}$: proof that `encryptedResults` is stored in leaf `0x4` of the state tree.
- private $\texttt{inclusionProof}^{\texttt{encryptionKey}}$: proof that `encryptionKey` is stored in leaf `0x3` of the state tree.
- private $\{\pi_j^{\text{dec}}\}_{j=1}^{n_f}$: per-field proofs of correct decryption.

Constraints

- *State commitment.* The encrypted results and the encryption public key match the values committed in the final state:

$$\mathsf{MT}^{\texttt{state}}.\mathsf{Verify}(\texttt{encryptedResults}, \texttt{0x4}, \texttt{inclusionProof}^{\texttt{encryptedResults}}, \texttt{stateRoot}),$$

$$\mathsf{MT}^{\texttt{state}}.\mathsf{Verify}(\texttt{encryptionKey}, \texttt{0x3}, \texttt{inclusionProof}^{\texttt{encryptionKey}}, \texttt{stateRoot}).$$

- *Correct decryption.* For every ballot field $j \in \{1, \dots, n_f\}$, the public result results_j is the decryption of the encrypted accumulator $\texttt{encryptedResults}_j$ under `encryptionKey`, as attested by π_j^{dec}:

$$\mathsf{DecVerify}_{\texttt{encryptionKey}}\big(\texttt{encryptedResults}_j,\ \text{results}_j,\ \pi_j^{\text{dec}}\big) = \texttt{true}.$$

- *Results range.* Each result lies in the valid plaintext range:

$$0 \le \text{results}_j \le t - 1, \qquad j = 1, \dots, n_f,$$

  where t is the order of the BabyJubjub prime-order subgroup (see Appendix A.1) and n_f denotes the number of ballot fields (see Section 5).

The per-field decryption proofs π_j^{dec} are produced during the tally decryption phase, where the key wardens jointly decrypt the aggregated results and attest, through a discrete-log-equality proof, that the decryption was performed correctly under the threshold key. By checking these proofs against the public key committed at leaf `0x3`, the circuit guarantees that the released tally corresponds exactly to the encrypted results accumulated in the final state, without ever exposing the decryption key inside the circuit.

4.6 Protocol flow

In this section we describe the overall flow of the DAVINCI voting protocol. An election is structured into five main phases: election setup, encryption key generation, voting period, tally decryption, and election finalization. Each phase specifies the interactions between the organizer, key wardens, voters, sequencers, and the smart contracts that coordinate the election.

4.6.1 Election setup

For each new election, the organizer performs the following steps:

1. Prepare the census data `censusData` and build the incremental Merkle tree $\mathsf{MT}^{\mathtt{census}}$ from it. Publish the full tree to IPFS, and record the resulting URI as `censusURI`.
2. Compute the census commitment $\mathtt{censusRoot} = \mathsf{MT}^{\mathtt{census}}.\mathsf{Root}()$.
3. Define the election details: the start time (`startTime`), the duration of the voting period (`duration`), the census model `censusOrigin` (see Section 4.3), and the ballot rules `ballotMode` (see Section 5).
4. Upload the election metadata (questions, options, description, etc.) to IPFS and record the resulting URI as `electionMetadata`.
5. Determine the process identifier `processID`, a 32-byte value derived deterministically from the organizer's Ethereum address, the account nonce, and a prefix encoding the chain identifier and the ElectionRegistry smart contract. Being deterministic, it can be computed in advance by the organizer, although it is ultimately recomputed and assigned by the contract upon registration.
6. Obtain the encryption public key `encryptionKey` for this process. The key is produced by the key wardens through the DKG protocol run for the identifier `processID`, so that no single party knows the corresponding secret key.
7. Register the process on-chain by submitting a transaction to the ElectionRegistry smart contract with the election parameters:

$$\mathsf{tx} = \big(\mathtt{censusOrigin},\ \mathtt{censusRoot},\ \mathtt{censusURI},\ \mathtt{electionMetadata},\ \mathtt{ballotMode},\ \mathtt{encryptionKey},\ \mathtt{startTime},\ \mathtt{duration}\big).$$

   Upon accepting this transaction, the ElectionRegistry contract initializes the reserved configuration leaves of the state tree $\mathsf{MT}^{\mathtt{state}}$ as `processID` at `0x0`, `ballotMode` at `0x2`, `encryptionKey` at `0x3`, `censusOrigin` at `0x6`, and the results accumulator `encryptedResults` at `0x4` initialized to the encryption of zero (see Section 4.4) and computes the initial state root

$$\mathtt{stateRoot} = \mathsf{MT}^{\mathtt{state}}.\mathsf{Root}().$$

Once this transaction is accepted, the election is publicly registered on-chain and, at its scheduled start time, the voting period can begin.

4.6.2 Encryption key generation

In this phase, the key wardens collectively generate the public encryption key (`encryptionKey`) that voters will use to encrypt their ballots. Since the process identifier `processID` is derived deterministically (see Section 4.6.1), it is known in advance, so the DKG ceremony for the election can be carried out before the process is registered on-chain.

Each key warden contributes to the ceremony and publishes a proof that its contribution is well-formed. The KeyManagement smart contract verifies these proofs, so that correct participation can be checked without revealing any secret. Once a threshold of valid contributions is collected, the joint public key `encryptionKey` is derived, while the corresponding secret key remains secret-shared among the wardens. In this way, no single party can decrypt on its own, and later tally decryption requires the collaboration of a threshold of wardens (see Section 4.6.4). The resulting `encryptionKey` is the key that the organizer includes when registering the election (see Section 4.6.1). The concrete DKG construction is given in Appendix A.7.

4.6.3 Voting period

In this phase, voters cast their ballots and send them to the sequencers, who collect and process them.

(a) Vote casting. Each voter holds an Ethereum key pair $(\mathsf{sk}, \mathsf{pk})$ with associated address `address` (see Appendix A.5). To cast a ballot, a voter does the following:

1. Retrieve the census membership proof `censusProof` to prove eligibility.

2. Fetch the parameters `encryptionKey`, `ballotMode`, and `processID` from the ElectionRegistry contract.
3. Select their ballot choice `ballot` following the rules of `ballotMode`.
4. Sample fresh randomness $k \in \mathbb{F}$ and encrypt the ballot using `encryptionKey`:

$$\texttt{encryptedBallot} = \mathsf{Enc}_{\texttt{encryptionKey}}(\texttt{ballot}; k). \tag{1}$$

5. Use their address `address` and the randomness k to compute a unique vote identifier, mapped into the reserved vote-identifier namespace (see Section 4.4):

$$\texttt{voteID} = N + \big(\mathsf{Poseidon}(\texttt{processID}, \texttt{address}, k) \bmod N\big). \tag{2}$$

6. Use the BallotCircuit from Section 4.5.1 to generate a ZK-SNARK proof that Eqs. (1) and (2) are correctly computed and that `ballot` satisfies the ballot rules of `ballotMode`:

$$\begin{aligned}\texttt{ballotProof} = \mathsf{P.Prove}(&\mathsf{BallotCircuit}, \mathsf{witness} = (\texttt{ballot}, k),\\ &\mathsf{PI} = (\texttt{processID}, \texttt{ballotMode}, \texttt{encryptionKey}, \texttt{address}, \texttt{weight}, \texttt{encryptedBallot}, \texttt{voteID})).\end{aligned}$$

7. Sign the vote identifier with their Ethereum secret key sk to authenticate the vote:

$$\texttt{signature} = \mathsf{S.Sign}_{\mathsf{sk}}(\texttt{voteID}).$$

8. Select a sequencer and submit the package

$$\texttt{vote} = [\texttt{processID}, \texttt{voteID}, \texttt{encryptedBallot}, \texttt{censusProof}, \texttt{ballotProof}, \texttt{signature}]$$

to the chosen sequencer.

(b) Vote batching. Upon receiving votes, a sequencer does the following:

1. Retrieve the election identifier `processID`, the current state root `stateRoot`, and the census root `censusRoot` from the ElectionRegistry smart contract, together with the associated data from the Ethereum data blobs.
2. Upon receiving a vote

$$\texttt{vote}_i = [\texttt{processID}, \texttt{voteID}_i, \texttt{encryptedBallot}_i, \texttt{censusProof}_i, \texttt{ballotProof}_i, \texttt{signature}_i],$$

recover the voter's public key $\mathsf{pk}_i = \mathsf{S.ExtractPublicKey}(\texttt{signature}_i)$.
3. For each $\texttt{vote}_i$, use the VerifierCircuit from Section 4.5.2 to generate a proof

$$\begin{aligned}\texttt{authenticationProof}_i = \mathsf{P.Prove}(&\mathsf{VerifierCircuit}, \mathsf{witness} = (\mathsf{pk}_i, \texttt{signature}_i, \texttt{ballotProof}_i),\\ &\mathsf{PI} = (\texttt{processID}, \texttt{ballotMode}, \texttt{encryptionKey}, \texttt{encryptedBallot}_i, \texttt{voteID}_i,\\ &\texttt{address}_i, \texttt{weight}_i)).\end{aligned}$$

This proof ensures that $\texttt{ballotProof}_i$ and $\texttt{signature}_i$ are valid and that $\texttt{address}_i$ is correctly derived from pk_i. The census membership proof $\texttt{censusProof}_i$ is not used here; it is retained for the state transition circuit (Section 4.5.4), where eligibility is enforced.
4. Batch a set of n authentication proofs $\{\texttt{authenticationProof}_i\}_{i=1}^{n}$ and verify them together using the AggregationCircuit from Section 4.5.3:

$$\begin{aligned}\texttt{aggregationProof} = \mathsf{P.Prove}(&\mathsf{AggregationCircuit}, \mathsf{witness} = (\{\texttt{authenticationProof}_i\}_{i=1}^{n}),\\ &\mathsf{PI} = (\{\texttt{encryptedBallot}_i\}_{i=1}^{n}, \{\texttt{voteID}_i\}_{i=1}^{n}, \{\texttt{address}_i\}_{i=1}^{n}, \texttt{processID},\\ &\texttt{ballotMode}, \texttt{encryptionKey})).\end{aligned}$$

This proof ensures that all $\texttt{authenticationProof}_i$ are valid and share the same global parameters `processID`, `ballotMode`, and `encryptionKey`.
5. Prepare the state transition data to be published in an Ethereum blob: pack the votes and results into a 4096-cell blob blobData and

- compute the data commitment $\mathsf{blobCommitment} = \mathsf{C.Commit}(\mathsf{blobData})$,
- derive the evaluation point $\mathsf{evalPoint} = \mathsf{Poseidon}(\texttt{processID}, \texttt{stateRoot}^{\texttt{old}}, \mathsf{blobCommitment})$,
- build the polynomial P from $\mathsf{blobData}$ and evaluate $y = P(\mathsf{evalPoint})$,
- produce the opening proof $\mathsf{openingProof}$ for the evaluation $(\mathsf{evalPoint}, y)$.

The consistency $y = P(\mathsf{evalPoint})$ between the blob and the on-chain state is proven inside the state transition circuit.

6. Generate the state transition proof with the $\mathsf{StateTransitionCircuit}$ from Section 4.5.4:

$$\begin{aligned}\texttt{stateProof} = \mathsf{P.Prove}(\mathsf{StateTransitionCircuit}, \mathsf{witness} = (&\texttt{aggregationProof}, \{\texttt{censusProof}_i\}_{i=1}^{n},\\ &\{\texttt{encryptedBallot}_i\}_{i=1}^{n}, k, \mathsf{openingProof}, \dots),\\ &\mathsf{PI} = (\texttt{censusRoot}, \texttt{stateRoot}^{\texttt{old}}, \texttt{stateRoot}^{\texttt{new}}, \mathsf{blobCommitment}, \dots)).\end{aligned}$$

   The complete input list is given in Section 4.5.4. This proof attests that the encrypted votes are correctly accumulated via ElGamal's homomorphic properties, that every processed vote corresponds to an eligible voter (via the census membership proofs), that vote overwrites are handled consistently, and the blob data matches the committed on-chain state.
7. Submit the state transition proof $\texttt{stateProof}$ and the new state root $\texttt{stateRoot}^{\texttt{new}}$ to the $\mathsf{ElectionRegistry}$ smart contract, while the full data is published in the Ethereum blobs.

Upon receiving the transaction, the smart contract checks that the submitted previous root matches the current on-chain state root and that $\texttt{stateProof}$ is valid before updating the state root to $\texttt{stateRoot}^{\texttt{new}}$. If the on-chain state root changed while the sequencer was preparing the batch (because another sequencer committed first), only the state transition proof must be regenerated, as noted in Section 4.5.4. Sequencers repeat this process until the voting deadline set by the organizer.

4.6.4 Tally decryption

After the voting period expires, the $\mathsf{ElectionRegistry}$ smart contract stops accepting new state updates, fixing the final state root and thus the aggregated encrypted results. A threshold of t out of n key wardens then cooperate to decrypt this aggregate: rather than reconstructing the election secret key, each participating warden publishes a partial decryption of the encrypted results together with a proof that its contribution is correct with respect to its key share. Once t valid partial decryptions are available, they are combined to recover the plaintext tally. Because only the aggregated results are decrypted and never the individual ballots, voter privacy is preserved throughout the election, and the secret key is never reconstructed by any single party. The resulting tally, together with a proof of correct decryption, is published on-chain during the finalization phase (see Section 4.6.5), where it is checked by the $\mathsf{ResultsCircuit}$ and the $\mathsf{ResultsVerifier}$ contract.

4.6.5 Election finalization

In this final phase, the organizer (or a sequencer) publishes the election results, together with a proof of their correct computation, on-chain.

1. Obtain the plaintext results $\{\text{results}_j\}_{j=1}^{n_f}$ and the corresponding decryption proofs $\{\pi_j^{\text{dec}}\}_{j=1}^{n_f}$ from the tally decryption phase (Section 4.6.4), i.e. the combination of the key wardens' partial decryptions of the aggregated encrypted results.
2. Retrieve the encrypted results accumulator $\texttt{encryptedResults}$ and the encryption key $\texttt{encryptionKey}$ from the final state, together with their Merkle inclusion proofs against the final state root $\texttt{stateRoot}$.
3. Use the $\mathsf{ResultsCircuit}$ from Section 4.5.5 to generate the results proof:

$$\begin{aligned}\texttt{resultsProof} = \mathsf{P.Prove}(\mathsf{ResultsCircuit}, \mathsf{witness} = (&\texttt{encryptedResults}, \texttt{encryptionKey},\\ &\texttt{inclusionProof}^{\texttt{encryptedResults}}, \texttt{inclusionProof}^{\texttt{encryptionKey}}, \{\pi_j^{\text{dec}}\}_{j=1}^{n_f}),\\ &\mathsf{PI} = (\texttt{stateRoot}, \{\text{results}_j\}_{j=1}^{n_f})).\end{aligned}$$

4. Submit the results and `resultsProof` to the ElectionRegistry smart contract. The contract verifies the proof with the ResultsVerifier, checks that the state root committed in the proof matches the current on-chain state root, and records the results, marking the process as finalized.

The election is considered finalized once the decrypted results are available on-chain. At this stage, both the results and the complete integrity of the process are permanently recorded and publicly auditable: anyone can recompute the on-chain commitments and check the ZK-SNARK proofs. In particular, the combination of the final state root, the verified decryption of the aggregated results, and the publicly verifiable ZK-SNARK state and results proofs guarantees the integrity of the entire election.

5 Ballot protocol

The ballot protocol provides a unified and parametric way to represent a wide range of voting systems within DAVINCI. Instead of designing a separate circuit for each voting rule, the protocol defines ballots as fixed-length arrays of integers, subject to a small set of configurable parameters. These parameters constrain the values that can appear in each field of the ballot and the aggregate properties of the ballot as a whole. By adjusting these parameters, the same circuit can implement approval voting, ranking, quadratic voting, multiple choice, and many other schemes. In the protocol we refer to the different configurations as the *ballot mode* (`ballotMode`). This abstraction has two key advantages. First, it allows a broad variety of voting systems to be supported with a minimal circuit design, avoiding conditional logic that would otherwise increase constraint counts. Second, it provides a common interface for tallying, since all ballots are aggregated into a single results array regardless of the voting mode.

Definition of parameters. Each ballot is defined as an array of integer values subject to a set of configurable parameters, illustrated in Fig. 7. These parameters are defined as follows:

- `numFields`: the maximum number of fields (i.e., options) in the ballot (by default, `numFields` $= 16$).
- `minValue`: the minimum value that each field in the ballot can take.
- `maxValue`: the maximum value that each field in the ballot can take.
- `uniqueValues`: a Boolean flag indicating whether all field values must be different (as in ranking systems).
- `costExponent`: the exponent applied when computing the cost of casting votes in a field. This parameter enables quadratic or higher-order voting rules.
- `minValueSum`: the minimum allowed total sum of a ballot, computed as $\sum_{i=1}^{\texttt{numFields}} v_i^{\texttt{costExponent}}$, where v_i are the field values.
- `maxValueSum`: the maximum allowed total sum of a ballot, computed as $\sum_{i=1}^{\texttt{numFields}} v_i^{\texttt{costExponent}}$, where v_i are the field values (e.g., to enforce a budget of credits).

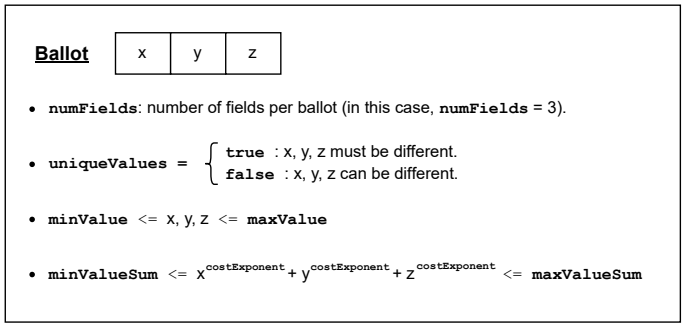

Fig. 7: Schematic representation of the parameters that define a ballot in DAVINCI.

Definition of constraints. Given a ballot with n fields and each field filled with a value v_i for $i = 1, \ldots, n$, we enforce the following constraints (captured in Fig. 7):

- `numFields` $= n$,
- If `uniqueValues` $=$ `true`, then $v_i \neq v_j$ for all $i \neq j$.
- `minValue` $\leq v_i \leq$ `maxValue` for all $i \in \{1, \ldots, n\}$.
- `minValueSum` $\leq \sum_{i=1}^{n} v_i^{\texttt{costExponent}} \leq$ `maxValueSum`.

General mechanism. A ballot is valid if and only if all the above constraints are satisfied. Invalid ballots are rejected at the circuit level and do not contribute to the tally. Valid ballots are aggregated into a results array, where each entry is the sum of all votes cast for the corresponding field across all voters. Voter weights, defined in the census tree (see Section 4.3), are taken into account when computing these sums. By default, all voters have the same weight, but the protocol also supports weighted voting, where different participants may contribute proportionally to their assigned voting power. In this case, `maxValueSum` reflects the maximum weight assigned to each voter, and the ballot and verifier circuits enforce that the correct weight is being used (see Sections 4.5.1 and 4.5.2 for further details).

Ballot modes. Ballot modes correspond to different voting systems, each adjusted with the variables above ballot configurations. For usability, the organizer does not need to set every parameter manually: instead, they simply select one of the predefined ballot modes (e.g., quadratic voting, ranking, approval), each corresponding to a fixed parameter configuration. Some representative modes are shown in Fig. 8, which illustrate the expressiveness of the ballot protocol. For example, in *approval voting*, each field is binary and voters can approve or reject multiple options simultaneously. In *ranking*, fields must form a permutation of the integers from 1 to N, enforcing uniqueness of values. In *quadratic voting*, voters allocate credits across options, and the cost is quadratic in the number of credits used, captured by setting `costExponent` $= 2$. Other systems such as single-choice or multiple-choice elections are special cases of the same framework. The figure also illustrates examples of voters' ballots, with the last column indicating whether each ballot is valid or invalid under the rules of the selected mode.

Name	Description	Example	numFields	minValue	maxValue	uniqueValues	costExponent	minValueSum	maxValueSum	Ballots example							
											1	2	3	4	5	totalValueSum	validVote
Approval voting	Multiple fields with only a 0-1 option	A referendum with 5 yes/no questions	5	0	1	FALSE	1	0	5	ballot 1	0	1	0	1	1	3	1
										ballot 2	1	1	1	1	1	5	1
										ballot 3	0	1	0	0	0	1	1
										total	**1**	**3**	**1**	**2**	**2**		**3**
											1	2	3	4	5	totalValueSum	validVote
Rating	Any field can be rated independently of the others	Satisfaction survey on 5 items, rated from 0 to 10 stars	5	0	10	FALSE	1	0	50	ballot 1	8	6	4	7	10	35	1
										ballot 2	5	4	6	12	4	31	0
										ballot 3	0	1	3	5	2	11	1
										total	**8**	**7**	**7**	**12**	**12**		**2**
											1	2	3	4	5	totalValueSum	validVote
Ranking	Number all options from 1 to N, using each number exactly once	Rank 5 destinations for the end-of-year school trip	5	1	5	TRUE	1	6	15	ballot 1	1	3	2	4	5	15	1
										ballot 2	1	5	3	4	5	18	0
										ballot 3	0	1	3	5	2	11	0
										total	**1**	**3**	**2**	**4**	**5**		**1**
											1	2	3	4	5	totalValueSum	validVote
Quadratic	Distribute M credits among N options (quadratically)	You have 12 credits to distribute among 5 options	5	0	12	FALSE	2	0	12	ballot 1	1	1	2	0	0	6	1
										ballot 2	3	0	0	0	2	13	0
										ballot 3	3	1	0	1	0	11	1
										total	**4**	**2**	**2**	**1**	**0**		**2**
											1	2	3	4	5	totalValueSum	validVote
Single choice	Select 1 among N options	Select a single winner from among 5 candidates	5	0	1	FALSE	1	1	1	ballot 1	1	0	0	0	0	1	1
										ballot 2	0	1	0	0	0	1	1
										ballot 3	0	1	1	0	0	2	0
										total	**1**	**1**	**0**	**0**	**0**		**2**
											1	2	3	4	5	totalValueSum	validVote
Multiple choice	Select M among N options	Select up to 3 finalists from 5 candidates	5	0	1	FALSE	1	0	3	ballot 1	1	0	1	1	0	3	1
										ballot 2	0	1	1	0	1	3	1
										ballot 3	1	2	3	0	0	6	0
										total	**1**	**1**	**2**	**1**	**1**		**2**

Fig. 8: Table showing various voting models using different ballot configurations. Each row corresponds to a voting system specified by a fixed configuration of the ballot parameters (`numFields`, `minValue`, `maxValue`, `uniqueValues`, `costExponent`, `minValueSum`, `maxValueSum`). The table also includes example ballots for each mode, illustrating how the constraints are enforced in practice. The last column indicates whether a ballot is valid (1) or invalid (0) under the rules of the selected mode.

6 Analysis

6.1 Security discussion

Based on the above principles, the protocol provides a number of concrete security properties that ensure the integrity, confidentiality, and verifiability of voting events. In what follows we discuss how DAVINCI achieves these properties, with particular emphasis on integrity, receipt-freeness, and ballot privacy, before turning to data availability, end-to-end verifiability, and robustness against quantum threats.

Integrity and soundness. The integrity of the election rests on the knowledge soundness of the ZK-SNARK proofs rather than on trust in any operator. No party (including a malicious sequencer) can advance the state from S_i to S_{i+1} without supplying a valid state transition proof, since the settlement contract rejects any update not accompanied by such a proof. Consequently, every counted ballot originates from an eligible census member and every accumulator update follows the protocol rules. Double voting is prevented *without* nullifiers: each voter is bound to a unique, census-authenticated slot in the state tree, so a second submission by the same voter overwrites their previous ballot at that slot rather than adding a new contribution to the tally. Vote identifiers, by contrast, are derived per ballot and therefore differ across submissions, which is precisely what enables individual verifiability without linking a voter to their ciphertext.

Receipt-freeness. Receipt-freeness prevents voters from proving to others how they voted, thereby mitigating coercion and vote-buying. In DAVINCI this is achieved through re-encryption, ballot overwrites, and randomized state updates.

- *Ballot re-encryption*: since our encryption scheme supports re-randomization, that is, a ciphertext can be refreshed with new randomness without changing the underlying message, sequencers re-encrypt the received ballots before committing them to the state Merkle tree. Under the decisional Diffie–Hellman (DDH) assumption on BabyJubjub, a re-randomized ciphertext is indistinguishable from a fresh encryption, so linking a submitted ciphertext to its stored counterpart is as hard as solving DDH.
- *Handling receipts*: because ballots are re-randomized, voters cannot generate a receipt by revealing the randomness k used in encryption, since the ciphertext on-chain no longer corresponds to their k. This blocks vote-selling and coercion.
- *Overwrites*: voters may cast a new ballot at any time, replacing their earlier submission. This ensures that even if coercion occurs, a voter can subsequently change their vote as many times as desired, preserving their ability to express their true choice. When an overwrite occurs, the sequencer subtracts the previous ballot and adds the new one to the results accumulator, and re-encrypts the updated ballot before storing it in the state tree.
- *Concealing overwrites*: to prevent observers from distinguishing overwrites from routine re-randomizations, sequencers periodically re-encrypt a random subset of stored ballots. This obfuscation ensures that overwrites remain indistinguishable from normal re-randomizations, strengthening receipt-freeness.

Privacy. Ballots remain secret even though encrypted ballots are publicly stored and processed. Ballot secrecy is preserved through ElGamal encryption, which allows votes to be aggregated without decryption. Because the encryption key is generated through a DKG and its secret counterpart is never reconstructed, no single party can decrypt individual ballots: the protocol only invokes threshold decryption on the final aggregate, never on each ballot separately. Under the assumption that at most $t-1$ key wardens are compromised, the shared secret key cannot be reconstructed and no individual ballot ciphertext can be decrypted. Note that, while the content of a ballot stays hidden, the protocol does reveal whether a given voter has cast a vote, that is, turnout is public, but the choice is not.

Quantum resistance. Quantum computers threaten discrete-logarithm-based cryptography such as ECDSA and ElGamal. For this reason, DAVINCI is designed in a modular way that would allow migration to post-quantum primitives in the longer term. Candidate replacements include lattice-based signature schemes such as CRYSTALS-Dilithium (ML-DSA) and Falcon, and the hash-based SPHINCS+ (SLH-DSA); lattice-based homomorphic encryption such as BGV or BFV; and ZK-STARKs for transparent, post-quantum ZK proofs.

Data availability. Ballots and state updates are stored in Ethereum blobs. Since these blobs may eventually be pruned from the blockchain, ensuring long-term data availability is an open problem. Possible solutions include decentralized storage networks or dedicated data-availability layers. This remains an active area of research.

End-to-end verifiability. Voters can check that their own ballot was included correctly, while anyone can audit the entire process to verify the final tally. Every voter can verify their ballot from casting to result computation (individual verifiability). Additionally, any third party can audit the election data to confirm results (universal verifiability) and verify that each vote comes from a uniquely registered voter (eligibility verifiability). Transparent cryptographic mechanisms make this possible.

6.2 Implementation

DAVINCI is implemented as a modular, fully open-source system: the smart contracts, the sequencer node, and the circuits are all publicly available at `https://github.com/vocdoni` (in particular `davinci-contracts`, `davinci-node`, and `davinci-circom`). Its components can be recombined or integrated with external services through adaptable interfaces, enabling redundancy and integration with third-party applications such as our voting-as-a-service APIs.

Technological stack and circuit compilation. The DAVINCI circuits are implemented with Circom [3] and gnark [9], and all use the Groth16 proof system [22] for its minimal proof size and constant-time verification. Each circuit is compiled over the elliptic curve best suited to its role (see Appendix A.1). The ballot circuit is compiled with Circom over BN254 for compatibility with Ethereum's precompiles; its threshold-ElGamal operations run over BabyJubjub, whose base field matches the BN254 scalar field. Ballot authentication and census membership are verified in the verifier circuit over BLS12-377, where the voter's ECDSA signature is checked through emulated SECP256K1 arithmetic, so the protocol can reuse standard Ethereum wallet keys. The resulting proofs are recursively combined in the aggregation circuit over BW6-761 before being verified back inside the BN254 state transition circuit. The final tally proof is likewise produced over BN254, since it is verified on-chain by the ResultsVerifier contract.

Circuit sizes. Concrete circuit sizes range from approximately 65K constraints for the ballot circuit to around 16M constraints for the state transition circuit, with the aggregation circuit dimensioned for batches of 60 ballots per transition.

Trusted setup. Because Groth16 relies on a circuit-specific common reference string (CRS, see Appendix A.8), the parameters for each circuit are generated through an MPC ceremony: a universal powers-of-tau phase, followed by a per-circuit phase that binds the parameters to each constraint system. As long as at least one participant in each ceremony discards their secret contribution, the resulting CRS is secure.

Implementation status. The current implementation includes all five circuits, the Ethereum smart contracts, and the complete state-transition pipeline. The DKG component is implemented but not yet integrated into the live protocol. The codebase has not yet undergone an independent security audit.

6.3 Performance evaluation

We evaluate DAVINCI along three axes: voter-side proving, sequencer proving, and on-chain verification, together with the data-availability savings from EIP-4844. All benchmarks use the protocol batch size of 60 ballots per state transition.

Voter proving. On consumer hardware (a modern laptop or smartphone), generating a ballot proof takes roughly 10 seconds in a browser, well within the latency a voter tolerates at casting time and without any specialized hardware.

Sequencer proving. The recursive aggregation and state transition circuits were benchmarked on batches of 60 ballots. A single sequencer running on a server with a consumer-grade GPU and at least 16GiB of RAM produces the final settlement proof in under 3 minutes; the same proof can be generated on a standard CPU with at least 32GiB of RAM in roughly 2.5× the time, keeping the sequencer entry barrier low. Proving parallelizes in two ways: about 85% of the per-batch work can be split across worker processes within a single sequencer, and multiple sequencers can advance independent batches concurrently. With 10 workers behind one GPU sequencer, the settlement proof is generated in under 32 seconds; two sequencers with 10 workers each fsustain a throughput of 200 settled votes per minute.

On-chain verification. Thanks to Groth16's succinctness, verifying the final settlement proof on Ethereum costs an essentially constant $\approx$480K gas, independent of the number of ballots in the batch.

Data availability. DAVINCI publishes the data needed to reconstruct each state transition through EIP-4844 blobs. Compared with calldata, blobs substantially reduce data-availability costs by separating large protocol data from permanent Ethereum state. In our implementation, the data behind a batch of 60 votes fits into a single 128KiB blob, the payload unit defined by EIP-4844.

Together, these results indicate that DAVINCI scales to large elections at predictable settlement costs and practical proving times. Deploying on an Ethereum L2 or an EVM-compatible sidechain would further reduce per-election cost and raise throughput, since the dominant bottleneck on sequencer parallelization is the block time at which successive state transitions can be settled.

7 Conclusions

We presented DAVINCI, a decentralized, E2E-verifiable voting protocol implemented as a specialized ZK rollup. By modeling elections as ZK-verified state transitions settled on Ethereum, DAVINCI combines public auditability with scalable off-chain execution while minimizing reliance on any single party. Ballot secrecy is achieved through threshold ElGamal encryption under a DKG-generated key, while ciphertext re-randomization and silent revoting strengthen protection against coercion and vote-selling.

To our knowledge, DAVINCI is the first system to combine a universal ballot abstraction, an open sequencer set, and fully proof-enforced election execution within a single rollup architecture. This enables a reusable voting infrastructure in which elections with different ballot formats and organizers can share the same settlement layer, verifier contracts, and proving pipeline.

Several directions remain open for future work. An important long-term direction is post-quantum migration, particularly through STARK-based proving systems and post-quantum signature and encryption schemes. Another is simplifying the proving architecture through general-purpose zkVM-based implementations, which may reduce development complexity and support larger proving batches. We also plan to integrate with real-world credential providers and conduct large-scale deployment experiments in realistic electoral settings

Acknowledgments

The authors would like to thank the following reviewers and contributors for their valuable feedback and support: all team members from the Vocdoni association, Jordi Baylina (Iden3 and Polygon), Adrià Massanet (Privacy Scaling Explorations, Ethereum Foundation), Arnaucube (0xPARC), Javier Herranz (Polytechnic University of Catalonia), Jordi Puiggali (Secrets Vault), and Carla Ràfols (Pompeu Fabra University).

References

1. Adida, B.: Helios: Web-based open-audit voting. In: 17th USENIX Security Symposium (USENIX Security 08). pp. 335–348. USENIX Association, San Jose, CA (2008), https://www.usenix.org/legacy/event/sec08/tech/full_papers/adida/adida.pdf

2. Baylina, J., Bellés, M.: Sparse Merkle trees (2019), https://docs.iden3.io/publications/pdfs/Merkle-Tree.pdf
3. Bellés-Muñoz, M., Isabel, M., Muñoz-Tapia, J.L., Rubio, A., Baylina, J.: Circom: A circuit description language for building zero-knowledge applications. IEEE Transactions on Dependable and Secure Computing (2023). https://doi.org/10.1109/TDSC.2022.3232813
4. Bellés-Muñoz, M., Whitehat, B., Baylina, J., Daza, V., Muñoz-Tapia, J.L.: Twisted edwards elliptic curves for zero-knowledge circuits. Mathematics **9**(23) (2021). https://doi.org/10.3390/math9233022, https://www.mdpi.com/2227-7390/9/23/3022
5. Benaloh, J., Naehrig, M., Pereira, O., Wallach, D.S.: ElectionGuard: a cryptographic toolkit to enable verifiable elections. In: 33rd USENIX Security Symposium (USENIX Security 24). pp. 5485–5502. USENIX Association, Philadelphia, PA (Aug 2024), https://www.usenix.org/conference/usenixsecurity24/presentation/benaloh
6. Benaloh, J., Tuinstra, D.: Receipt-free secret-ballot elections (extended abstract). In: Proceedings of the Twenty-Sixth Annual ACM Symposium on Theory of Computing (STOC). pp. 544–553 (1994)
7. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: The KECCAK SHA-3 submission (2011), https://keccak.team/files/Keccak-submission-3.pdf
8. Blake, I.F., Seroussi, G., Smart, N.P.: Elliptic curves in cryptography. Cambridge University Press, USA (1999)
9. Botrel, G., Piellard, T., El Housni, Y., Kubjas, I., Tabaie, A.: Consensys/gnark. https://doi.org/10.5281/zenodo.5819104, https://doi.org/10.5281/zenodo.5819104
10. Bowe, S., Chiesa, A., Green, M., Miers, I., Mishra, P., Wu, H.: ZEXE: enabling decentralized private computation. In: 2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18-21, 2020. pp. 947–964. IEEE (2020). https://doi.org/10.1109/SP40000.2020.00050, https://doi.org/10.1109/SP40000.2020.00050
11. Brown, D.R.L.: SEC 2: Recommended elliptic curve domain parameters. In: Standards for efficient cryptography 2 (SEC 2) (2010), https://www.secg.org/sec2-v2.pdf
12. Buterin, V.: Minimal anti-collusion infrastructure. Ethereum Research Forum (ethresear.ch) (2019), https://ethresear.ch/t/minimal-anti-collusion-infrastructure/5413
13. Buterin, V., Feist, D., Loerakker, D., Kadianakis, G., Garnett, M., Taiwo, M., Dietrichs, A.: EIP-4844: Shard blob transactions, Ethereum Improvement Proposals, no. 4844, [online serial] (February 2022), https://eips.ethereum.org/EIPS/eip-4844
14. Canetti, R.: Universally composable security: a new paradigm for cryptographic protocols. In: Proceedings 42nd IEEE Symposium on Foundations of Computer Science. pp. 136–145 (2001). https://doi.org/10.1109/SFCS.2001.959888
15. Chaum, D.L.: Untraceable electronic mail, return addresses, and digital pseudonyms. Communications of the ACM **24**(2), 84–88 (1981)
16. Cortier, V., Gaudry, P., Glondu, S.: Belenios: A simple private and verifiable electronic voting system. In: Foundations of Security, Protocols, and Equational Reasoning. LNCS, vol. 11565, pp. 214–238. Springer, Cham (2019)
17. Doan, T.V.T., Pereira, O., Peters, T.: Threshold receipt-free single-pass eVoting. In: Electronic Voting – E-Vote-ID 2024. LNCS, vol. 15014. Springer (2024)
18. El Housni, Y., Guillevic, A.: Optimized and secure pairing-friendly elliptic curves suitable for one layer proof composition. In: Krenn, S., Shulman, H., Vaudenay, S. (eds.) Cryptology and Network Security. pp. 259–279. Springer International Publishing, Cham (2020)
19. Gennaro, R., Gentry, C., Parno, B., Raykova, M.: Quadratic span programs and succinct nizks without pcps. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 626–645. Springer (2013)
20. Glaeser, N., Seres, I.A., Zhu, M., Bonneau, J.: Cicada: A framework for private non-interactive on-chain auctions and voting. Cryptology ePrint Archive, Paper 2023/1473 (2023)
21. Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., Schofnegger, M.: Poseidon: A new hash function for zero-knowledge proof systems. In: 30th USENIX Security Symposium (USENIX Security 21). pp. 519–535 (2021)
22. Groth, J.: On the size of pairing-based non-interactive arguments. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 305–326. Springer (2016)
23. Gurkan, K., Wei Jie, K., Whitehat, B.: Semaphore: Zero-knowledge signaling on Ethereum. ZKProof Community Proposal, Workshop 3 (2020), https://semaphore.pse.dev/whitepaper-v1.pdf
24. Hirt, M., Sako, K.: Efficient receipt-free voting based on homomorphic encryption. In: Advances in Cryptology – EUROCRYPT 2000. LNCS, vol. 1807, pp. 539–556. Springer (2000)
25. Housni, Y.E., Connor, M., Guillevic, A.: EIP-3026: BW6-761 curve operations [DRAFT], Ethereum Improvement Proposals, no. 3026, [online serial] (October 2020), https://eips.ethereum.org/EIPS/eip-3026
26. Johnson, D., Menezes, A., Vanstone, S.: The elliptic curve digital signature algorithm (ECDSA). Int. J. Inf. Secur. **1**(1), 36–63 (August 2001). https://doi.org/10.1007/s102070100002, https://doi.org/10.1007/s102070100002

27. Kampa, A., Escrich, P., Bellés-Muñoz, M., Baig, R.: NI-DKG: Non-interactive distributed key generation using blockchain and zero-knowledge proofs. Cryptology ePrint Archive, Paper 2026/552 (2026), https://eprint.iacr.org/2026/552
28. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: International conference on the theory and application of cryptology and information security. pp. 177–194. Springer (2010)
29. Kirillov, D., Korkhov, V., Petrunin, V., Makarov, M., Khamitov, I.M., Dostov, V.: Implementation of an E-voting scheme using Hyperledger Fabric permissioned blockchain. In: Computational Science and Its Applications – ICCSA 2019. LNCS, vol. 11620, pp. 509–521. Springer, Cham (2019)
30. McCorry, P., Shahandashti, S.F., Hao, F.: A smart contract for boardroom voting with maximum voter privacy. In: Financial Cryptography and Data Security. LNCS, vol. 10322, pp. 357–375. Springer, Cham (2017)
31. Merkle, R.C.: A digital signature based on a conventional encryption function. In: Conference on the theory and application of cryptographic techniques. pp. 369–378. Springer (1987)
32. Noack, J., Bormet, L., et al.: Shutter network: Private transactions from threshold cryptography. Cryptology ePrint Archive, Paper 2024/1981 (2024)
33. Rarimo: Freedom tool: Privacy-preserving passport-based voting. Whitepaper, Rarimo (2024), https://docs.rarimo.com/freedom-tool/
34. Reuters: India's 2024 general election sees record participation with 642 million votes cast. https://www.reuters.com/world/india/india-poll-panel-says-642-mln-voters-cast-ballots-general-election-2024-06-03/ (2024), accessed 2026-01-22
35. Sutikno, S., Surya, A., Effendi, R.: An implementation of elgamal elliptic curves cryptosystems. In: IEEE. APCCAS 1998. 1998 IEEE Asia-Pacific Conference on Circuits and Systems. Microelectronics and Integrating Systems. Proceedings (Cat. No.98EX242). pp. 483–486 (1998). https://doi.org/10.1109/APCCAS.1998.743829
36. Vlasov, A., hujw77: EIP-2539: BLS12-377 curve operations [DRAFT], Ethereum Improvement Proposals, no. 2539, [online serial] (February 2020), https://eips.ethereum.org/EIPS/eip-2539
37. WhiteHat, B., Bellés, M., Baylina, J.: ERC-2494: Baby Jubjub elliptic curve [DRAFT], Ethereum Improvement Proposals, no. 2494, [online serial] (January 2020), https://eips.ethereum.org/EIPS/eip-2494
38. Wood, G., et al.: Ethereum: A secure decentralised generalised transaction ledger. Ethereum project yellow paper **151**(2014), 1–32 (2014)

A Cryptographic primitives

In this section, we present the cryptographic primitives used in the protocol described in Section 4. We first introduce elliptic curves as the underlying algebraic setting, and then describe the following building blocks, each tied to a specific role in the system: hash functions and Merkle trees for commitments to structured data; digital signatures for voter authentication; encryption schemes for ballot confidentiality; DKG for decentralizing trust in the election keys; and zero-knowledge proofs (ZK-SNARKs) for verifiability of computations. Standard primitives are cited rather than re-derived; we spell out only the operations and instantiations specific to DAVINCI.

A.1 Elliptic curves

DAVINCI relies on multiple elliptic curves to ensure interoperability with Ethereum, compatibility with available cryptographic primitives, and efficient in-circuit operations. On the one hand, SECP256K1 [11] is used because Ethereum public keys are elements of this curve. As DAVINCI assumes each voter holds a standard Ethereum address, cryptographic operations such as digital signatures and identity verification rely on SECP256K1 to match the Ethereum ecosystem. On the other hand, BN254 [38] is chosen because it is the curve supported by Ethereum's precompiled contracts for ZK proof verification, which makes it the optimal choice for verifying SNARK proofs on-chain with minimal gas cost. Then, BabyJubjub [4] is used as an inner curve for elliptic curve operations within arithmetic circuits. Finally, BLS12-377 [10] and BW6-761 [18] are used to enable recursive proof composition. More specifically, BLS12-377 acts as the inner curve for constructing proofs, while BW6-761 serves as the outer curve that verifies those proofs within larger ZK-SNARK circuits. This pairing enables succinct verification and composability of ZK-SNARKs within other ZK-SNARKs, which we use for DAVINCI's aggregation and state transition logic. Below, we give the details of these curves.

Parameters.

- $p =$ `0xfffffffffffffffffffffffffffffffffffffffffffffffffffffffefffffc2f` (256-bit prime).
- $q =$ `0xfffffffffffffffffffffffffffffffebaaedce6af48a03bbfd25e8cd0364141` (256-bit prime).
- $r =$ `0x30644e72e131a029b85045b68181585d97816a916871ca8d3c208c16d87cfd47` (254-bit prime).
- $s =$ `0x30644e72e131a029b85045b68181585d2833e84879b9709143e1f593f0000001` (254-bit prime).
- $t =$ `0x60c89ce5c263405370a08b6d0302b0bab3eedb83920ee0a677297dc392126f1` (251-bit prime).
- $u =$ `0x122e824fb83ce0ad187c94004faff3eb926186a81d14688528275ef8087be41707ba638e584e9190 3cebaff25b423048689c8ed12f9fd9071dcd3dc73ebff2e98a116c25667a8f8160cf8aeeaf0a437e69 13e6870000082f49d00000000008b` (761-bit prime).
- $v =$ `0x01ae3a4617c510eac63b05c06ca1493b1a22d9f300f5138f1ef3622fba094800170b5d4430000000 8508c00000000001` (377-bit prime).
- $w =$ `0x12ab655e9a2ca55660b44d1e5c37b00159aa76fed00000010a11800000000001` (253-bit prime).

Elliptic curve groups.

- SECP256K1 curve: $E^{\mathsf{SEC}}/\mathbb{F}_p$ defined by equation $E^{\mathsf{SEC}} : y^2 = x^3 + 7$, with group $\mathbb{G}^{\mathsf{SEC}}$ of prime order q.
- BN254 curve: $E^{\mathsf{BN}}/\mathbb{F}_r$ defined by equation $E^{\mathsf{BN}} : y^2 = x^3 + 3$, with subgroups $\mathbb{G}_1^{\mathsf{BN}}, \mathbb{G}_2^{\mathsf{BN}}$ of prime order s, and an efficiently computable pairing $e : \mathbb{G}_1^{\mathsf{BN}} \times \mathbb{G}_2^{\mathsf{BN}} \to \mathbb{G}_T^{\mathsf{BN}}$, where $\mathbb{G}_T^{\mathsf{BN}} \subset \mathbb{F}_{s^{12}}$ and has order s.
- BabyJubjub curve: $E^{\mathsf{BJ}}/\mathbb{F}_s$ defined by equation $E^{\mathsf{BJ}} : 168700x^2 + y^2 = 1 + 168696x^2y^2$, with subgroup $\mathbb{G}^{\mathsf{BJ}}$ of prime order t.
- BW6-761 curve: $E^{\mathsf{BW}}/\mathbb{F}_u$ defined by equation $E^{\mathsf{BW}} : y^2 = x^3 - 1$, with subgroups $\mathbb{G}_1^{\mathsf{BW}}, \mathbb{G}_2^{\mathsf{BW}}$ of prime order v, and an efficiently computable pairing $e : \mathbb{G}_1^{\mathsf{BW}} \times \mathbb{G}_2^{\mathsf{BW}} \to \mathbb{G}_T^{\mathsf{BW}}$, where $\mathbb{G}_T^{\mathsf{BW}} \subset \mathbb{F}_{v^6}$ and has order v as well.
- BLS12-377 curve: $E^{\mathsf{BLS}}/\mathbb{F}_v$ defined by equation $E^{\mathsf{BLS}} : y^2 = x^3 + 1$, with subgroups $\mathbb{G}_1^{\mathsf{BLS}}, \mathbb{G}_2^{\mathsf{BLS}}$ of prime order w, and an efficiently computable pairing $e : \mathbb{G}_1^{\mathsf{BLS}} \times \mathbb{G}_2^{\mathsf{BLS}} \to \mathbb{G}_T^{\mathsf{BLS}}$, where $\mathbb{G}_T^{\mathsf{BLS}} \subset \mathbb{F}_{w^{12}}$ and has order w.

Finite fields.

- $\mathbb{F}_p$: base field of the SECP256K1 curve.
- $\mathbb{F}_q$: scalar field of $\mathbb{G}^{\mathsf{SEC}}$.
- $\mathbb{F}_r$: base field of the BN254 curve.
- $\mathbb{F}_s$: scalar field of $\mathbb{G}_1^{\mathsf{BN}}, \mathbb{G}_2^{\mathsf{BN}}, \mathbb{G}_T^{\mathsf{BN}}$ and base field of the BabyJubjub curve.
- $\mathbb{F}_t$: scalar field of $\mathbb{G}^{\mathsf{BJ}}$.
- $\mathbb{F}_u$: base field of the BW6-761 curve.
- $\mathbb{F}_v$: scalar field of $\mathbb{G}_1^{\mathsf{BW}}, \mathbb{G}_2^{\mathsf{BW}}, \mathbb{G}_T^{\mathsf{BW}}$ and base field of the BLS12-377 curve.
- $\mathbb{F}_w$: scalar field of $\mathbb{G}_1^{\mathsf{BLS}}, \mathbb{G}_2^{\mathsf{BLS}}, \mathbb{G}_T^{\mathsf{BLS}}$.

Generators.

We denote by G^{SEC} the generator of $\mathbb{G}^{\mathsf{SEC}}$ as defined in [11], G^{BN} the generator of $\mathbb{G}_1^{\mathsf{BN}}$ as defined in [38], G^{BJ} the generator of $\mathbb{G}^{\mathsf{BJ}}$ as defined in [37], G^{BW} the generator of $\mathbb{G}_1^{\mathsf{BW}}$ as defined in [25], and G^{BLS} the generator of $\mathbb{G}_1^{\mathsf{BLS}}$ as defined in [36].

- $G^{\mathsf{SEC}} =$ (`0x79be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798`, `0x483ada7726a3c4655da4fbfc0e1108a8fd17b448a68554199c47d08ffb10d4b8`).
- $G^{\mathsf{BN}} =$ (`0x01`, `0x02`).
- $G^{\mathsf{BJ}} =$ (`0x0b9feffffffffaaabffffffffffffffefffffffffffffffffffffffffffffff`, `0x17ffffffffffffffffffffffffffffffffffffffffffffffffffebaaedce6af48a03bbfd25e8cd0364141`).
- $G^{\mathsf{BW}} =$ (`0x1075b020ea190c8b277ce98a477beaee6a0cfb7551b27f0ee05c54b85f56fc779017ffac15520ac11dbfcd294c2e746a17a54ce47729b905bd71fa0c9ea097103758f9a280ca27f6750dd0356133e82055928aca6af603f4088f3af66e5b43d`, `0x58b84e0a6fc574e6fd637b45cc2a420f952589884c9ec61a7348d2a2e573a3265909f1af7e0dbac5b8fa1771b5b806cc685d31717a4c55be3fb90b6fc2cdd49f9df141b3053253b2b08119cad0fb93ad1cb2be0b20d2a1bafc8f2db4e95363`).
- $G^{\mathsf{BLS}} =$ (`0x008848defe740a67c8fc6225bf87ff5485951e2caa9d41bb188282c8bd37cb5cd5481512ffcd394eeab9b16eb21be9ef`, `0x01914a69c5102eff1f674f5d30afeec4bd7fb348ca3e52d96d182ad44fb82305c2fe3d3634a9591afd82de55559c8ea6`).

A.2 Hash functions

DAVINCI uses two hash functions: Keccak256 [7] for Ethereum address derivation and message hashing under ECDSA, and Poseidon [21] for every in-circuit computation.

Keccak256. Keccak256 is the standard hash function used by Ethereum and is employed in DAVINCI to compress messages for signing and to derive Ethereum addresses from public keys over $\mathbb{G}^{\mathsf{SEC}}$. It maps arbitrary-length bitstrings to 256-bit digests, $\mathsf{Keccak} : \{0,1\}^* \to \{0,1\}^{256}$. As it is not SNARK-friendly, it is used only in the authentication circuit, where verifying Ethereum-compatible signatures requires reproducing the message hash computed by Ethereum clients (see Section 4.5.2).

Poseidon. Poseidon is a SNARK-optimized hash function designed for efficient evaluation inside arithmetic circuits. It is used in DAVINCI for computing commitments, vote identifiers, and Merkle tree roots, and for refreshing the re-encryption randomness during state transitions. We write $\mathsf{Poseidon} : \mathfrak{F}_s \to \mathbb{F}_s$, where $\mathfrak{F}_s$ denotes tuples of $\mathbb{F}_s$-elements of any length and $\mathbb{F}_s$ is the field described in Section A.1.

A.3 Merkle trees

DAVINCI uses two Merkle-tree variants as hash-based accumulators: a *sparse Merkle tree* (SMT) for the election state, and an *incremental Merkle tree* (IMT) for the census. Both are instantiated with the Poseidon hash function over the BN254 scalar field, and only the root of each tree is published on-chain, so that large datasets can be committed and verified with minimal on-chain storage.

Sparse Merkle tree (state tree). Following [2], the state tree is a fixed-depth binary tree in which each leaf encodes a key–value pair and empty leaves are represented explicitly, supporting both inclusion and non-inclusion proofs. The key determines a unique root-to-leaf path independent of insertion order, which makes membership and emptiness checks cheap to verify inside a ZK-SNARK. DAVINCI uses an SMT of depth 64 for the state tree, whose key space is partitioned into disjoint regions for configuration parameters, encrypted ballots, and vote identifiers (see Section 4.4).

Incremental Merkle tree (census tree). The census is maintained as an IMT, a binary tree optimized for sequential insertion: voters are appended at consecutive leaf positions rather than at key-derived paths, so the tree grows only as needed and appending a voter recomputes hashes along a single path. Each leaf stores a voter's identifier and weight. Concretely, the Ethereum address and weight packed into a single field element (see Section 4.3) and only the root `censusRoot` is published on-chain.

Tree operations. To support state transitions and in-circuit verification, each tree MT exposes the following operations:

- $\mathsf{MT.Root}()$: returns the current Merkle root of the tree.
- $\mathsf{MT.Insert}(\mathit{path}, \mathsf{leaf})$: inserts a new leaf leaf at the position defined by the given path.
- $\mathsf{MT.Update}(\mathsf{leaf}_{\mathsf{old}}, \mathsf{leaf})$: replaces an existing leaf.
- $\mathsf{MT.MembershipProof}(\mathsf{leaf})$: returns a Merkle proof of inclusion for the given leaf.
- $\mathsf{MT.NonMembershipProof}(\mathsf{leaf})$: returns a proof that the given leaf is not included in the tree.
- $\mathsf{MT.Verify}(\mathsf{leaf}, \mathit{path}, \mathsf{Proof}, \mathsf{root})$: returns `true` if and only if Proof shows that leaf is stored at *path* under the claimed root.

These operations let sequencers evolve the state off-chain while keeping every update independently verifiable on-chain and within ZK circuits. Additional details on the tree structures and their leaf encodings are given in Sections 4.3 and 4.4.

A.4 Commitment scheme

DAVINCI commits to the data published in each Ethereum blob using the KZG polynomial commitment scheme [28], the same scheme that underlies EIP-4844; it therefore reuses Ethereum's BLS12-381 structured reference string rather than requiring a dedicated ceremony. The blob payload is encoded as a polynomial P; the sequencer commits to P and later proves a single evaluation, which lets the state transition circuit check consistency between the blob data and the on-chain state (see Section 4.5.4). We use two operations:

- $\mathsf{C.Commit}(P)$: outputs a succinct commitment C to the polynomial P.
- $\mathsf{C.Verify}(C, z, y, \pi)$: returns `true` if and only if π is a valid proof that $P(z) = y$ for the polynomial committed in C.

A.5 Digital signature scheme

DAVINCI uses the elliptic curve digital signature algorithm (ECDSA) [26] over the SECP256K1 curve, so that voters authenticate with standard Ethereum wallets without managing additional keys. We use ECDSA in its standard form and refer to [26] for the full algorithms; here we only fix the notation used throughout the protocol:

- $\mathsf{S.Sign}_{\mathsf{sk}}(\mathsf{message})$: produces a signature σ on message $\mathsf{message}$ under the secret key sk.
- $\mathsf{S.Verify}_{\mathsf{pk}}(\mathsf{message}, \sigma)$: returns `true` if σ is a valid signature on $\mathsf{message}$ under the public key pk.
- $\mathsf{S.ExtractPublicKey}(\mathsf{message}, \sigma)$: recovers the signer's public key pk from a message and its signature; this is the operation Ethereum exposes as `ecrecover`.

An Ethereum address is derived from a public key as the low-order 20 bytes of its Keccak digest, $\texttt{address} = \mathsf{low\text{-}20}\big(\mathsf{Keccak}(\mathsf{pk})\big)$. Because SECP256K1 is defined over a 256-bit field that differs from the native field of our ZK-SNARK circuits, signature verification is performed in-circuit through a specialized gadget that emulates SECP256K1 arithmetic (see Section 4.5.2).

A.6 Encryption scheme

To preserve ballot secrecy while enabling tallying, DAVINCI employs a threshold variant of the ElGamal cryptosystem instantiated over the BabyJubjub curve [35]. This scheme offers two properties crucial for the protocol: *additive homomorphism*, which allows the aggregation of encrypted votes without decryption, and *re-encryption*, which enables ciphertext randomization without altering the underlying plaintext. Together, these properties allow sequencers to tally votes while preventing voters from producing receipts that could be used for coercion or vote selling. The corresponding public key is generated collectively by the key wardens via the distributed key generation protocol of Section A.7, ensuring that no single entity can decrypt ballots unilaterally.

Encryption and decryption. Given a message m, the ElGamal encryption algorithm maps it into a group element M of $\mathbb{G}^{\text{BJ}}$ and outputs a ciphertext as follows:

- $\mathsf{Enc}_{\mathsf{pk}}(m; k \in \mathbb{F}_t)$:
  1. Map m to $\mathbb{G}^{\text{BJ}}$ via $M = m \cdot G^{\text{BJ}}$.
  2. Compute $C_1 = k \cdot G^{\text{BJ}} \in \mathbb{G}^{\text{BJ}}$.
  3. Compute $C_2 = k \cdot \mathsf{pk} + M \in \mathbb{G}^{\text{BJ}}$.
  4. Output $\mathsf{ciphertext} = (C_1, C_2) \in (\mathbb{G}^{\text{BJ}})^2$.
- $\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ciphertext})$:
  1. Parse $\mathsf{ciphertext} = (C_1, C_2)$.
  2. Compute $K = \mathsf{sk} \cdot C_1 \in \mathbb{G}^{\text{BJ}}$.
  3. Output $M = C_2 - K$.

Since the plaintext message space is small, recovering m from M is efficient: although the mapping $m \mapsto M$ is not generally invertible, it can be reversed by brute-force search or optimized techniques such as baby-step giant-step [8].

Homomorphic addition and re-encryption. ElGamal encryption is additively homomorphic. Given two ciphertexts (C_1, C_2) and (C_1', C_2'), their component-wise sum $(C_1 + C_1',\ C_2 + C_2')$ is a valid ciphertext that decrypts to the sum of the two underlying messages, which allows sequencers to aggregate encrypted ballots directly. Re-encryption exploits the same property by adding an encryption of zero to a ciphertext, yielding a fresh ciphertext of the same message under new randomness and making it computationally infeasible to link the re-encrypted ballot with the original submission.

- $\mathsf{EncAdd}_{\mathsf{pk}}(\mathsf{ciphertext} \in (\mathbb{G}^{\text{BJ}})^2, \mathsf{ciphertext}' \in (\mathbb{G}^{\text{BJ}})^2)$:
  1. Parse $\mathsf{ciphertext} = (C_1, C_2) \in (\mathbb{G}^{\text{BJ}})^2$.
  2. Parse $\mathsf{ciphertext}' = (C_1', C_2') \in (\mathbb{G}^{\text{BJ}})^2$.
  3. Output $(C_1 + C_1', C_2 + C_2') \in (\mathbb{G}^{\text{BJ}})^2$.
- $\mathsf{ReEnc}_{\mathsf{pk}}(\mathsf{ciphertext} \in (\mathbb{G}^{\text{BJ}})^2; k \in \mathbb{F}_t)$:
  1. Parse $\mathsf{ciphertext} = (C_1, C_2) \in (\mathbb{G}^{\text{BJ}})^2$.
  2. Compute $\mathsf{ciphertext}' = \mathsf{Enc}_{\mathsf{pk}}(0; k)$.
  3. Output $\mathsf{EncAdd}_{\mathsf{pk}}(\mathsf{ciphertext}, \mathsf{ciphertext}')$.

In the protocol, re-encryption is applied not only to newly submitted ballots but also to ballots already stored in the state tree. By refreshing the randomness of both new and existing ciphertexts, sequencers ensure that it is indistinguishable whether a ballot has been overwritten or merely re-randomized. This mechanism is essential for receipt-freeness: voters cannot produce a verifiable receipt of their choice, and adversaries cannot detect or prove whether a particular vote has been replaced (see Section 6).

A.7 Distributed key generation scheme

To ensure that no single entity controls the decryption key, DAVINCI employs a DKG protocol to jointly derive the threshold ElGamal encryption key pair used for ballot encryption. Participants who contribute to the ceremony are called *key wardens*. The outcome is a collective public key (`encryptionKey`) that is published on-chain and used by voters to encrypt their ballots, while the corresponding private key is secret-shared among the key wardens so that only a threshold t out of n of them can later collaborate to decrypt the tally. Unlike classical DKG protocols in which shares are exchanged in the clear, our construction distributes encrypted shares together with ZK-SNARK proofs of correctness, so that the KeyManagement smart contract can verify each contribution without learning any of the underlying secrets and slash misbehaving participants. We use the non-interactive distributed key generation protocol of [27] and refer the reader to that work for the full construction and its security analysis.

A.8 Zero-knowledge proof systems

Zero-knowledge succinct non-interactive arguments of knowledge (ZK-SNARKs) are the backbone of DAVINCI's integrity guarantees: voters prove that their encrypted ballots are well-formed and rule-compliant without revealing their choices, and sequencers prove that vote aggregation and state transitions were carried out correctly. For a circuit $\mathcal{C}$ with public inputs PI and witness $\mathsf{witness}$, we write:

- $\mathsf{P.Prove}(\mathcal{C}, \mathsf{witness}, \mathsf{PI})$: outputs a proof π that the prover knows a witness satisfying $\mathcal{C}$ for the given public inputs, using the circuit's proving key.
- $\mathsf{P.Verify}(\pi, \mathsf{PI})$: returns `true` if and only if π is valid for PI, using the circuit's verification key.

All DAVINCI circuits use the Groth16 proof system [22], chosen for its constant-size proofs and cheap on-chain verification. Although they share the proving system, the circuits are instantiated over different curves according to their role: the ballot circuit over BN254, the authentication (verifier) circuit over BLS12-377, the aggregation circuit over BW6-761, and the state transition and results circuits over BN254 (the latter because their proofs are verified on-chain through Ethereum's native precompiles). Further details on the circuits are provided in Section 4.5 and a summary of the instantiations can be found in Fig. 2.